



DISEÑO DE MODELO ARQUITECTÓNICO ASÍNCRONO PARA LA IMPLEMENTACIÓN DE SERVICIOS DE COMPUTACIÓN SIN SERVIDOR EN LA INFRAESTRUCTURA DE NUBE PRIVADA DE IFX NETWORKS

Autor

BREITNER ENRIQUE GONZÁLEZ ANGARITA

Director

PhD. PAULO ALONSO GAONA GARCIA

Doctor en Ingeniería de la Información y del Conocimiento

Co-Director

LIOR HAREL

Chief Information Officer – IFX Networks

FACULTAD DE INGENIERIA

MAESTRÍA EN CIENCIAS DE LA INFORMACIÓN Y LAS COMUNICACIONES

ÉNFASIS EN INGENIERÍA DE SOFTWARE

mayo de 2025

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

A mi hijo,
mi mayor inspiración y motivo de esfuerzo constante.

Tu sonrisa y tu amor me recuerdan cada día la importancia de avanzar, crecer y
nunca rendirme.

Este logro es tan mío como tuyo, porque en cada página de este trabajo está el amor,
la paciencia y la fuerza que me das.

Gracias por enseñarme el verdadero significado del amor incondicional.

RESUMEN

La computación sin servidor o serverless es un modelo arquitectónico donde los desarrolladores de software no tienen que preocuparse por la infraestructura o por la gestión de la infraestructura en la que se ejecuten sus aplicaciones, centrándose solo en desarrollar la lógica de negocio de sus soluciones.

IFX Networks es una compañía que ofrece a sus clientes servicios de telecomunicaciones, Data Centers (IaaS), Cloud pública y privada, entre otros, sin embargo, los servicios de computación sin servidor aun no son implementados debido al costo que representa el uso de este tipo de servicios con otros proveedores de nube pública. En este sentido se hizo necesario plantear a través de este proyecto el desarrollo de un modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks, que permita al centro de desarrollos y automatizaciones de la compañía aprovechar su infraestructura utilizando este modelo como estrategia de desarrollo de software moderno para su equipo de fábrica de software, optimizando costos y logrando eficiencia en la construcción de software.

El propósito del proyecto es diseñar un modelo asíncrono para la implementación de servicios sin servidor (serverless) en la nube privada de IFX Networks, para esto en la capa de presentación se utilizó una arquitectura de Micro Frontend con control de acceso basado en roles (RBAC) el cual ofrece múltiples beneficios como seguridad, modularidad, autonomía y escalabilidad (Kurapati, 2024), proveyendo a la solución de un modelo robusto de autenticación y autorización compartidos. De manera adicional entre los componentes de Frontend y Backend se utilizó un sistema de seguridad a nivel de conectividad a través del uso de Web Application Firewall (WAF) buscando establecer una barrera confiable para el tráfico del código de las funciones serverless en la red interna de la solución, la cual la protege contra ataques como la inyección SQL, cross-site scripting (XSS), entre otros (Anwar, Abdullah, & Pastore, 2021), lo anterior mediante un enfoque asíncrono con un middleware con protocolo AMQP (Advanced Message Queuing Protocol), mejorando la escalabilidad y la tolerancia a fallos del sistema (Ćatović, Buzadžija, & Lemeš, 2022).

PALABRAS CLAVE: Arquitectura Cloud, Serverless, BaaS, FaaS, Cloud Computing, computación en la nube.

ABSTRACT

Serverless computing is an architectural model where software developers do not have to worry about the infrastructure or the management of the environment where their applications run, focusing solely on developing the business logic of their solutions.

IFX Networks is a company that provides its clients with telecommunications services, Data Centers (IaaS), and public and private cloud services, among others. However, serverless computing services have not yet been implemented due to the costs associated with using such services through public cloud providers. In this context, this project proposes the development of an asynchronous architectural model for the implementation of serverless computing services within IFX Networks' private cloud infrastructure. This model enables the company's development and automation center to leverage its infrastructure by adopting this approach as a modern software development strategy for its software factory team, optimizing costs and improving software construction efficiency.

The goal of this project is to design an asynchronous model for the implementation of serverless services in IFX Networks' private cloud. To achieve this, a Micro Frontend architecture with Role-Based Access Control (RBAC) was used in the presentation layer, offering multiple benefits such as security, modularity, autonomy, and scalability (Kurapati, 2024). This provides the solution with a robust shared authentication and authorization model. Additionally, a Web Application Firewall (WAF) was implemented between the Frontend and Backend components to establish a reliable security barrier for serverless function code traffic within the internal network. This protects the solution from attacks such as SQL injection, cross-site scripting (XSS), among others (Anwar, Abdullah, & Pastore, 2021). The architecture follows an asynchronous approach using middleware with the AMQP (Advanced Message Queuing Protocol), improving the system's scalability and fault tolerance (Ćatović, Buzadžija, & Lemeš, 2022).

KEYWORDS: Cloud Architecture, Serverless, BaaS, FaaS, Cloud Computing.

Contenido

INTRODUCCIÓN	11
1. PROBLEMA DE INVESTIGACIÓN	13
1.1 PLANTEAMIENTO DEL PROBLEMA	13
1.2 DEFINICIÓN DEL PROBLEMA	14
1.3 PREGUNTA DE INVESTIGACIÓN	14
2. OBJETIVOS	15
2.1 OBJETIVO GENERAL	15
2.2 OBJETIVOS ESPECÍFICOS	15
3. JUSTIFICACIÓN	16
4. MARCO DE REFERENCIA	17
4.1. NUBE PRIVADA	17
4.2. VIRTUALIZACIÓN	17
4.3. HIPERVISOR	18
4.4. MÁQUINAS VIRTUALES	18
4.5. DOCKER	19
4.6. KUBERNETES	19
4.7. FUNCIONES COMO SERVICIO (FAAS)	20
4.8. DIFERENCIAS ENTRE FAAS Y SERVERLESS	20
4.9. BACKEND COMO SERVICIO (BAAS)	21
4.10. CONTENEDORES COMO SERVICIO (CAAS)	21
4.11. ARQUITECTURAS DISTRIBUIDAS	21
4.12. ARQUITECTURAS ASÍNCRONAS	22
4.13. HERRAMIENTAS Y TECNOLOGÍAS SERVERLESS	23
4.13.1. KNATIVE	23
4.13.2. APACHE OPENWHISK	24
4.13.3. OPENFAAS	24
4.13.4. FN PROJECT	24
5. PROYECTOS DE REFERENCIA	25
5.1. APLICACIÓN DE TECNOLOGÍAS Y ARQUITECTURAS SERVERLESS PARA EL DESARROLLO IOT	25
5.2. ON-PREMISES INFRASTRUCTURE FOR SERVERLESS COMPUTING	26

5.3.	SERVERLESS COMPUTING, FUNCIONES COMO SERVICIO (FAAS) PARA EL SOPORTE DE CARGAS COMPUTACIONALES EN LA NUBE.....	26
6.	METODOLOGÍA.....	28
6.1.	FASE DE ANÁLISIS	29
6.2.	FASE DE DISEÑO	30
6.3.	FASE DE IMPLEMENTACIÓN	31
6.4.	FASE VALIDACIÓN Y PRUEBAS DEL MODELO.....	31
7.	DESARROLLO DE LA PROPUESTA	33
7.1.	FASE DE ANÁLISIS	34
7.1.1.	REQUISITOS FUNCIONALES	34
7.1.2.	REQUISITOS TÉCNICOS Y DE SEGURIDAD	35
7.1.3.	MATRIZ COMPARATIVA DE MODELOS EXISTENTES Y SELECCIÓN DE ELEMENTOS CLAVE PARA EL MODELO PROPUESTO.....	36
7.2.	FASE DE DISEÑO	37
7.2.1.	DOCUMENTACIÓN TÉCNICA DE IFX NETWORKS	38
7.2.2.	DIAGRAMA DE CASOS DE USO	39
7.2.3.	DIAGRAMA DE CONTEXTO.....	39
7.2.4.	DIAGRAMA DE PROCESO.....	40
7.2.5.	MOCKUP	41
7.2.6.	PROTOTIPO PLATAFORMA WEB	42
7.2.7.	DISEÑO DE LA ARQUITECTURA	43
7.2.7.1.	DIAGRAMA DE CLASES DE IFX FAAS.....	44
7.2.7.2.	DIAGRAMA DE DESPLIEGUE DE IFX FAAS	46
7.2.7.3.	DIAGRAMA MODELO ENTIDAD RELACIÓN MODELO IFX FAAS	48
7.2.7.4.	DIAGRAMA DE INFRAESTRUCTURA MODELO IFX FAAS.....	48
7.2.7.5.	DIAGRAMA DE CONTENEDORES MODELO IFX FAAS	49
7.2.7.6.	VISTA ARQUITECTÓNICA DEL MODELO IFX FAAS.....	50
7.2.8.	HERRAMIENTAS, LENGUAJES DE DESARROLLO Y ENTORNO PARA EL MODELO IFX FAAS	51
7.3.	FASE DE IMPLEMENTACIÓN MODELO IFX FAAS	51
7.3.1.	CÓDIGO FUENTE - FUNCIONANDO Y DOCUMENTADO	51
7.3.1.1.	VENTANA CON EL LISTADO DE PROYECTOS SERVERLESS PARA LOS EQUIPOS DEL CENTRO DE DESARROLLO DE IFX.....	52
7.3.1.2.	CREACIÓN DE FUNCIONES	52
7.3.1.3.	ASISTENTE PARA GENERAR CÓDIGO A PARTIR DE UN ASISTENTE CON IA.....	54
7.3.1.4.	DESPLEGAR LA FUNCIÓN	58

7.3.1.5.	VALIDAR LA FUNCIÓN DESPLEGADA	59
7.4.	FASE DE PRUEBAS Y VALIDACIÓN DEL MODELO	60
7.4.1.	EJECUCIÓN DE PRUEBAS	60
7.4.1.1.	INFORMACIÓN GENERAL DE LAS PRUEBAS DE RENDIMIENTO	60
7.4.1.2.	ESTADÍSTICAS POR ENDPOINT	61
7.4.1.3.	EVALUACIÓN DE LOS TIEMPOS DE RESPUESTA DE LOS MÉTODOS	62
7.4.1.4.	HILOS ACTIVOS A LO LARGO DE LA PRUEBA	62
4.	CONCLUSIONES	64
5.	REFERENCIAS BIBLIOGRÁFICAS	65

Tabla de Ilustraciones

FIGURA 1. SERVIDORES Y VIRTUALIZACIÓN. FUENTE: TOMADO DE (CENTRIA, 2025)	18
FIGURA 2. ARQUITECTURA MÁQUINAS VIRTUALES VS DOCKER. FUENTE: (FIXED BUFFER, 2020).....	19
FIGURA 3. MODELO DE DESPLIEGUE CLÁSICO DE APLICACIONES VS CONTENEDORES. FUENTE: (KUBERNETES, 2022)	20
FIGURA 4. MODELOS CENTRALIZADOS VS DISTRIBUIDOS. FUENTE: (JUNG KANG & JIHYOUN, 2013)	21
FIGURA 5. MODELO DE REFERENCIA DE UNA ARQUITECTURA ASÍNCRONA. FUENTE: (MICROSOFT LEARN, 2025) ...	22
FIGURA 6. METODOLOGÍA DE DESARROLLO PARA LA IMPLEMENTACIÓN DE MODELO ARQUITECTÓNICO FUENTE: ELABORACIÓN PROPIA	28
FIGURA 7. DIAGRAMA DE CASO DE USO. FUENTE: ELABORACIÓN PROPIA	39
FIGURA 8. DIAGRAMA DE CONTEXTO. FUENTE: ELABORACIÓN PROPIA	40
FIGURA 9. DIAGRAMA DE PROCESO BPMN. FUENTE: ELABORACIÓN PROPIA.....	41
FIGURA 10. MOCKUP DE LA PLATAFORMA. FUENTE: ELABORACIÓN PROPIA	42
FIGURA 11. PROTOTIPO PLATAFORMA WEB. FUENTE: ELABORACIÓN PROPIA	42
FIGURA 12. PROTOTIPO CREAR FUNCIÓN. FUENTE: ELABORACIÓN PROPIA	43
FIGURA 13. DIAGRAMA DE CLASES DE IFX FAAS PARTE 1. FUENTE: ELABORACIÓN PROPIA	44
FIGURA 14. DIAGRAMA DE CLASES DE IFX FAAS PARTE 2. FUENTE: ELABORACIÓN PROPIA	45
FIGURA 15. DIAGRAMA DE DESPLIEGUE GENÉRICO MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA.....	47
FIGURA 16. DIAGRAMA DE DESPLIEGUE DETALLADO MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA	47
FIGURA 17. DIAGRAMA MER MODELO ENTIDAD RELACIÓN MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA ..	48
FIGURA 18. DIAGRAMA DE INFRAESTRUCTURA MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA ..	49
FIGURA 19. DIAGRAMA DE CONTENEDORES MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA	50
FIGURA 20. MODELO ARQUITECTÓNICO IFX FAAS. FUENTE: ELABORACIÓN PROPIA	51
FIGURA 21: VENTANA PRINCIPAL IFX FAAS. FUENTE: ELABORACIÓN PROPIA.....	52
FIGURA 22: MODULO CON EL LISTADO DE FUNCIONES DE UN PROYECTO. FUENTE: ELABORACIÓN PROPIA.	52
FIGURA 23: FORMULARIO PARA LA CREACIÓN DE FUNCIONES. FUENTE: ELABORACIÓN PROPIA.	53
FIGURA 24: EDITOR DE CÓDIGO DE FUNCIONES. FUENTE: ELABORACIÓN PROPIA.	53
FIGURA 25: ASISTENTE CON IA PARA LA GENERACIÓN DE CÓDIGO. FUENTE: ELABORACIÓN PROPIA.	54
FIGURA 26: RESPUESTA DEL ASISTENTE CON IA PARA LA GENERACIÓN DE CÓDIGO. FUENTE: ELABORACIÓN PROPIA	54
FIGURA 27: VALIDACIÓN DEL CÓDIGO CON IA. FUENTE: ELABORACIÓN PROPIA	55
FIGURA 28: RESULTADO DE ASISTENTE QUE VALIDA CÓDIGO CUANDO NO HAY ERRORES. FUENTE: ELABORACIÓN PROPIA.	55
FIGURA 29: RESPUESTA A ESCENARIOS CON ERRORES - 1. FUENTE: ELABORACIÓN PROPIA.	56
FIGURA 30: RESPUESTA A ESCENARIOS CON ERRORES - 2. FUENTE: ELABORACIÓN PROPIA.	56
FIGURA 31: RESPUESTA A ESCENARIOS CON ERRORES - 3. FUENTE: ELABORACIÓN PROPIA.	57
FIGURA 32: RESPUESTA A ESCENARIOS CON ERRORES - 4. FUENTE: ELABORACIÓN PROPIA.	57

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

FIGURA 33: DESPLIEGUE DE UNA FUNCIÓN - 1. FUENTE: ELABORACIÓN PROPIA.	58
FIGURA 34: DESPLIEGUE DE UNA FUNCIÓN - 2. FUENTE: ELABORACIÓN PROPIA.	58
FIGURA 35: URL DE LA FUNCIÓN DESPLEGADA. FUENTE: ELABORACIÓN PROPIA.	59
FIGURA 36: PRUEBA DE CONSUMO DE LA FUNCIÓN DESPLEGADA. FUENTE: ELABORACIÓN PROPIA.....	59
ILUSTRACIÓN 37. RESULTADOS DE LAS PRUEBAS MODELO IFX FAAS. FUENTE: IMAGEN TOMADA DE LOS RESULTADOS JMETER.....	61
FIGURA 38. TIEMPOS DE RESPUESTA A LO LARGO DE LA PRUEBA. FUENTE: HERRAMIENTA JMETER	62
FIGURA 39. HILOS ACTIVOS A LO LARGO DE LA PRUEBA. FUENTE: HERRAMIENTA JMETER.....	63

Tabla de Tablas

TABLA 1. LISTADO DE REQUISITOS FUNCIONALES. FUENTE: ELABORACIÓN PROPIA	35
TABLA 2. REQUISITOS TÉCNICOS Y DE SEGURIDAD. FUENTE: ELABORACIÓN PROPIA.....	36
TABLA 3. MATRIZ COMPARATIVA SERVICIOS FAAS FUENTE: ELABORACIÓN PROPIA CON BASE EN LAS FUENTES CONSULTADAS.....	37
TABLA 4. DESCRIPCIÓN DE COMPONENTES MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA	46
TABLA 5. PATRONES DE DISEÑO IMPLEMENTADOS MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA.....	46
TABLA 6. RESULTADOS DE LAS PRUEBAS DEL MODELO IFX FAAS. FUENTE: ELABORACIÓN PROPIA	60
TABLA 7. ESTADÍSTICAS DEL RESULTADO DE LAS PRUEBAS DE ENDPOINTS COMPILAR FUNCIÓN. FUENTE: ELABORACIÓN PROPIA A PARTIR DE RESULTADOS DE JMETER.....	61
TABLA 8. ESTADÍSTICAS DEL RESULTADO DE LAS PRUEBAS DE ENDPOINTS CREAR FUNCIÓN. FUENTE: ELABORACIÓN PROPIA A PARTIR DE RESULTADOS DE JMETER.	61

INTRODUCCIÓN

El interés y adopción de computación en la nube al interior de las compañías paso de ser una novedad, a convertirse en una necesidad, en donde no solo es necesario alcanzar los objetivos comerciales de las empresas sino que además las compañías buscan centrar sus esfuerzos en un uso sostenible de este tipo de servicios, esta tendencia se ve impulsada por la responsabilidad que tiene el sector de las TIC al generar entre el 1,8% y el 3,9% de las emisiones globales de gases de efecto invernadero (Danave, 2024).

La computación sin servidor o serverless es un paradigma de computación en la nube que le permite a los desarrolladores centrar sus esfuerzos en escribir código, dejando de lado problemas asociados a la gestión de servidores o infraestructura necesaria para el despliegue de sus aplicaciones, lo que les permite ser más ágiles al momento de entregar funcionalidades con el menor esfuerzo técnico y con un bajo acoplamiento, por lo que se espera que la computación sin servidor experimente un importante crecimiento anual compuesto (CAGR) de aproximadamente el 23,17% entre 2023 y 2028 (Danave, 2024).

Este tipo de modelos generalmente se encuentran disponibles a través de proveedores de nube publica como AWS (AWS Lambda), Google Cloud (Google Cloud Functions) o Azure (Azure Functions), sin embargo, el uso e implementación de este tipo de modelos sobre infraestructura de nube propia o privada no ofrecen todos los componentes de seguridad y escalabilidad para aquellas empresas que cuenten con infraestructura de nube privada propia y deseen utilizar este tipo de tecnologías con alternativas de bajo costo.

Las arquitecturas y modelo síncronos enfrentan grandes desafíos ya que al estar basados en comunicación a través de servicios web y operaciones HTTP vía Rest APIs, producen problemas de alto acoplamiento entre sus componentes, en donde aplicaciones enteras pueden verse afectadas si alguno de estos servicios independientes se ve afectado o comprometido (Billawa, y otros, 2022).

Este trabajo tiene como propósito la creación de un modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor dentro de la infraestructura de nube privada de la compañía IFX Networks. Compañía dedicada a proveer servicios de nube, que sirva como base para mejorar la eficiencia y sostenibilidad de los desarrollos implementados por los equipos de la fábrica de software de IFX.

Este proyecto se desarrolla bajo metodología en cascada aplicando fases secuenciales de análisis, diseño, implementación y pruebas, a partir de las cuales se logra definir el modelo propuesto con artefactos creados con base en mejores prácticas como la automatización, la calidad continua, la seguridad y la sostenibilidad, alineado a tendencias y practicas más relevantes usadas en la actualidad en la industria del software. Adicional, el modelo desarrollado cuenta con la interoperabilidad con el componente de inteligencia artificial propio de IFX Networks.

El modelo propuesto es probado mediante herramientas tecnológicas con ejecución de escenarios de 120 pruebas técnicas de carga y estrés en un periodo de tiempo de 60 segundos, resultados favorables de acuerdo con el entorno actual de IFX Networks. A futuro este servicio puede ser ofrecido a clientes de la industria que estén interesados en dar un paso hacia la computación sin servidor.

En cuanto a la estructura del presente documento, inicialmente se aborda el planteamiento del problema, los objetivos y la justificación de este proyecto, seguido de un apartado que detalla el marco de referencia y la literatura relacionada con otros proyectos y componentes técnicos que sustenta teóricamente la propuesta. Posteriormente, se expone la metodología en cascada aplicada para el desarrollo del modelo, y finalmente, el desarrollo de la propuesta y los resultados obtenidos en la evaluación del modelo IFX FaaS, los cuales evidencian la aplicabilidad y pertinencia de la propuesta frente a los retos actuales de la compañía IFX Networks en temas de computación sin servidor.

1. PROBLEMA DE INVESTIGACIÓN

Los servicios en la nube, entregados a demanda a través de Internet ofrecen un acceso rápido, fácil y escalable a servicios de cómputo, manejados en su mayoría por operadores de nube como Amazon Web Service (AWS), Microsoft Azure, o Google Cloud (GCP), así mismo el paradigma de computación sin servidor presenta una perspectiva de crecimiento junto a estos proveedores de nube (Danave, 2024).

La computación sin servidor o serverless es un concepto y modelo arquitectónico de nube relativamente nuevo, pero que ha avanzado muy rápidamente en los últimos años (WoSC6, 2020). Sin embargo, el crecimiento acelerado de esa tecnología apalancada por esos grandes proveedores de nube, deja sin muchas alternativas a quienes deseen utilizarla sin la limitante de quedar atados a un proveedor específico e incluso sin la posibilidad de usarla sin depender de ninguno de ellos (Zhao, Benomar, Pfandzeltery, & Georgantas, 2022).

En este aspecto IFX Networks a pesar de contar con infraestructura propia y tener la capacidad de ofrecer servicios de nube privada, aun no cuenta con un modelo que le permita hacer uso de computación sin servidor o serverless sin tener que recurrir a la contratación de servicios externos.

1.1 PLANTEAMIENTO DEL PROBLEMA

Gracias al incremento en el uso de computación en la nube, los proveedores de servicios cloud están teniendo que adaptarse a las nuevas necesidades de un mercado en constante cambio y crecimiento (Weins, Cloud computing trends and statistics: Flexera 2023 State of the Cloud Report, 2023). Sin embargo, las empresas con nube privada no cuentan con un modelo arquitectónico que le permita ofrecer servicios de computación sin servidor (serverless), y de esta forma estar alineado con las últimas tendencias tecnológicas, las cuales están siendo ampliamente utilizadas por otros proveedores de computación en la nube (Winzinger & Wirtz, 2019).

Se podría afirmar que las empresas con infraestructura propia no aprovechan todas las ventajas que ofrece la tecnología de computación sin servidor aun sabiendo que existe un mercado cada vez más creciente entorno a este tipo de estrategias (Castro, Ishakian, Muthusamy, & Slominski, 2019), lo que genera una pérdida de participación en el mercado por parte de IFX. Por lo tanto, se hace necesario el diseño de un modelo arquitectónico asíncrono que permita la implementación de servicios de computación sin servidor en infraestructura de nube privada de IFX Networks.

1.2 DEFINICIÓN DEL PROBLEMA

De acuerdo con aspectos tratados en el planteamiento del problema, pueden encontrarse diferentes modelos de computación sin servidor a través de los cuales se podría establecer una solución a la problemática antes planteada, y de esta manera formular la siguiente pregunta de investigación ¿Cómo el diseño de un modelo arquitectónico asíncrono permite la implementación de servicios de computación sin servidor en infraestructura de nube privada de IFX Networks?

1.3 PREGUNTA DE INVESTIGACIÓN

¿Qué aspectos de otros modelos arquitectónicos para la implementación de servicios serverless pueden ser reutilizables, con el propósito de conceptualizar un nuevo modelo que se adapte a las necesidades y características de infraestructura de nube privada de IFX Networks?

¿Cuáles protocolos y procesos de seguridad se deben aplicar al nuevo modelo de arquitectura serverless, para las fases de autenticación y autorización entre los componentes?

¿Qué aspectos se deben aplicar para el diseño y la documentación del modelo conceptual de una arquitectura de servicios sin servidor en infraestructura de nube privada?

¿Qué mecanismos son aplicables para verificar el funcionamiento la arquitectura propuesta?

2. OBJETIVOS

2.1 OBJETIVO GENERAL

Diseñar un modelo arquitectónico asíncrono que permita la implementación de servicios de computación sin servidor en infraestructura de nube privada de IFX Networks.

2.2 OBJETIVOS ESPECÍFICOS

- Identificar, analizar y evaluar propuestas para el diseño de un modelo arquitectónico que permita la implementación de servicios de tipo Serverless en infraestructura de nube privada de IFX Networks.
- Definir los protocolos y procesos de seguridad para las fases de autenticación y autorización entre los componentes.
- Diseñar y documentar un modelo conceptual de una arquitectura asíncrona que permita la implementación y uso de servicios sin servidor en infraestructura de nube privada de IFX Networks.
- Verificar el modelo de la arquitectura de software diseñada, a través de un prototipo serverless.

3. JUSTIFICACIÓN

El paradigma de computación sin servidor o serverless, es una tendencia emergente en el uso de computación en la nube. Existen múltiples factores que han acelerado este tipo de modelos de desarrollo de software. Por un lado, tenemos la popularidad que han adquirido arquitecturas empresariales que implementan contenedores y microservicios, lo que ha significado un cambio en los modelos tradicionales de desarrollo de software. Por otro lado, tenemos fenómenos sociales como el COVID-19, el cual ha llevado a que las empresas se vean obligadas a incrementar el uso de canales digitales para el desarrollo de sus actividades diarias y de ahí que también se evalúen arquitecturas que permitan la disminución de costos de operación, como es el caso de los servicios serverless.

Desde la perspectiva de un proveedor de servicios cloud, este cambio de paradigma ofrece una oportunidad de crecimiento, el cual deberá estar alineado con las necesidades y las tendencias del mercado. Las arquitecturas serverless, a diferencia de otros modelos de servicios cloud, les ofrece a los clientes una reducción de costos de implementación, así como un modelo de programación simplificado, que permite la ejecución de fragmentos de código nativo el cual podrá ejecutar funcionalidades específicas de acuerdo con la demanda que estos presenten, pagando únicamente por el tiempo de uso de estos servicios.

A diferencia de modelos como PaaS, los modelos de computación sin servidor le permiten al cliente transferir sus preocupaciones operativas como la gestión de servidores, la escalabilidad, el mantenimiento, y la administración en general, al proveedor de servicios serverless. Estos modelos de servicios sin servidor son el siguiente paso de la computación en la nube, sin embargo, aquellas empresas que cuenta con infraestructura propia y modelos de nube privada no pueden seguir el ritmo de los grandes proveedores de nube en cuanto al uso de tipo de tecnologías.

De esta forma, el diseño de un modelo arquitectónico para la implementación de servicios de computación sin servidor les permitirá a aquellas compañías con infraestructura propia, acceder al uso de estos servicios de última generación.

4. MARCO DE REFERENCIA

Este capítulo está centrado en hacer un análisis bibliográfico de aquellos términos, modelos y tecnologías que permitan la implementación de computación sin servidor utilizando la infraestructura de IFX y que pueda ser utilizada por los desarrolladores de la fábrica de software, es decir, se excluye aquellas herramientas o tecnologías que necesiten de proveedores externos como AWS Lambda, Azure Functions, Google Functions o cualquier otro relacionado.

4.1. NUBE PRIVADA

Hace referencia a un entorno de computación en la nube dedicado a una organización, este entorno consta de recursos como CPU, almacenamiento que se aprovisionan bajo demanda a través de un portal de autogestión, a este modelo también se le conoce como nube interna o corporativa. Existen diferentes tipos de soluciones de nube privada tales como nube privada en las instalaciones, nube privada administrada y nube privada virtual (Amazon AWS, 2025).

4.2. VIRTUALIZACIÓN

La virtualización es un modelo tecnológico basado en software que permite simular la funcionalidad del hardware lo que permite que un servidor físico pueda distribuir sus recursos de hardware en diferentes entornos, es decir, esta tecnología permite que múltiples máquinas virtuales con diferentes sistemas operativos puedan ejecutarse de manera independiente a partir de una sola máquina física (RedHat, 2025).

En la figura 1 se presenta el modelo tecnológico de servidores y virtualización.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

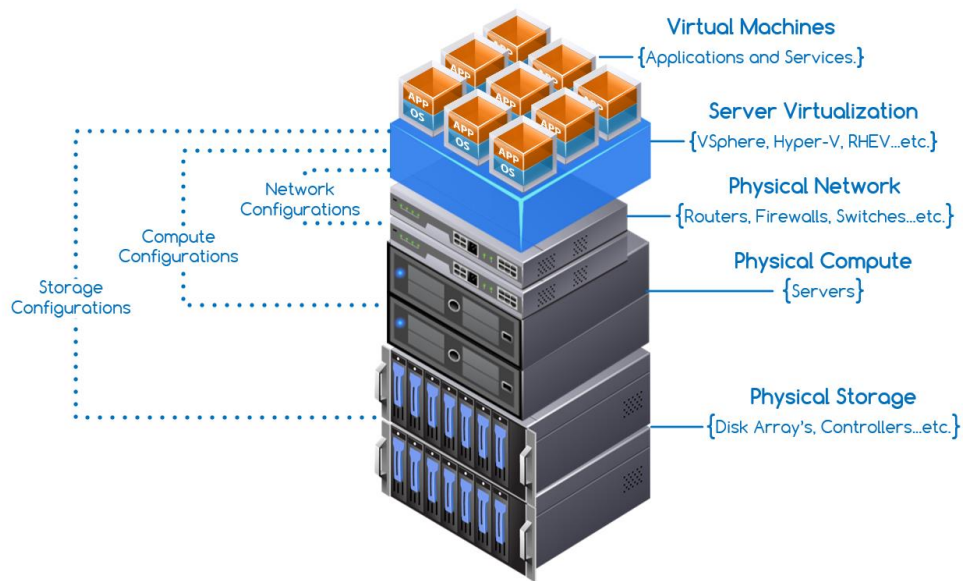


Figura 1. Servidores y virtualización. Fuente: Tomado de (Centria, 2025)

4.3. HIPERVISOR

Software que permite la distribución de los recursos de hardware que se asignan a las diferentes máquinas virtuales, esta tecnología hace posible la virtualización. Un hipervisor actúa como un supervisor de máquinas virtuales (RedHat, 2025).

4.4. MÁQUINAS VIRTUALES

Una máquina virtual (VM) es un contenedor de software aislado que contiene un sistema operativo y que se ejecuta en un servidor físico sobre una capa virtualizada. Cada VM es autónoma e completamente independiente, lo que le permite ejecutar varios sistemas operativos en un único servidor físico (Wmware, 2025).

4.5. DOCKER

Tecnología de código abierto que permite a los desarrolladores empaquetar, ejecutar, actualizar y gestionar sus aplicaciones en contenedores (Susnjara & Smalley, 2025).

La figura 2, muestra gráficamente la comparación entre arquitectura de máquinas virtuales frente a arquitectura Docker.

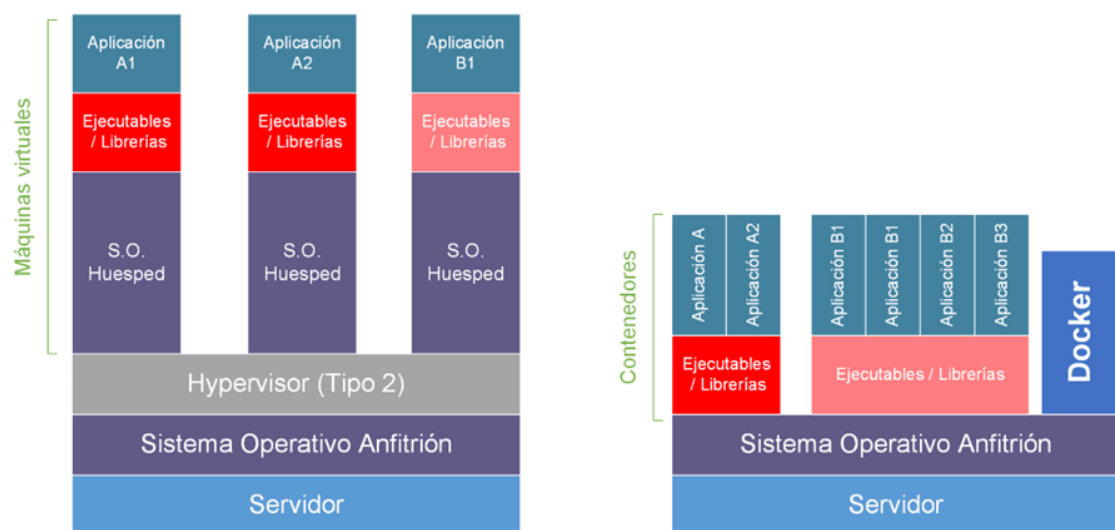


Figura 2. Arquitectura Máquinas Virtuales vs Docker. Fuente: (Fixed Buffer, 2020)

4.6. KUBERNETES

Plataforma de código abierto diseñada para la automatizar la implementación, el escalado y la gestión de aplicaciones en contenedores. Esta plataforma permite administrar y orquestar las cargas de trabajo de los servicios empaquetados en contenedores (Kubernetes, 2022).

La figura 3 presenta el modelo de despliegue clásico de aplicaciones versus contenedores observando sus diferencias en administración y orquestación.

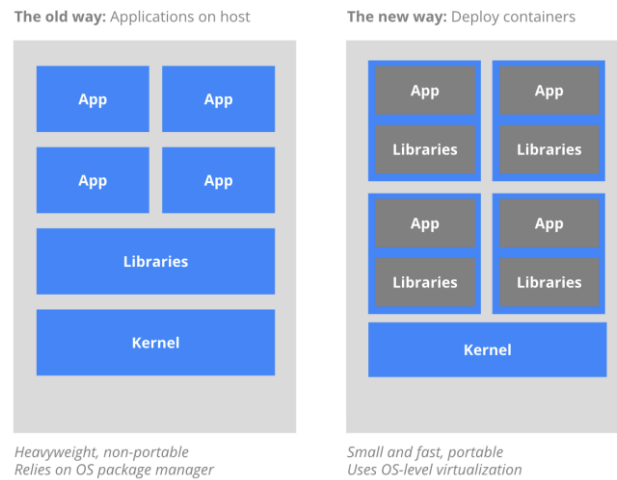


Figura 3. Modelo de despliegue clásico de aplicaciones vs contenedores. Fuente: (Kubernetes, 2022)

4.7. FUNCIONES COMO SERVICIO (FAAS)

Modelo de computación en la nube que permite a los desarrolladores ejecutar funciones individuales en respuesta a eventos específicos sin necesidad de gestionar la infraestructura subyacente.

4.8. DIFERENCIAS ENTRE FAAS Y SERVERLESS

La diferencia principal entre FaaS (Function as a Service) y Serverless radica en el alcance y enfoque de cada uno, en ambas el proveedor administra completamente la infraestructura, sin embargo en serverless se abarca una gama más amplia de servicios administrados sin servidor es decir bases de datos, almacenamiento, APIs, entre otros, en cambio en FaaS solo se enfoca en la ejecución de código en forma de funciones, Es decir todas las plataformas FaaS son serverless pero no todo lo que es serverless es FaaS (Kidd & Wickramasinghe, 2021).

4.9. BACKEND COMO SERVICIO (BAAS)

Es un modelo de servicio en la nube que permite a los desarrolladores de aplicaciones web y móviles centrarse en la capa de presentación y les brinda la posibilidad de conectar sus aplicaciones a recursos como almacenamiento en la nube, bases de datos, servicios de analítica, gestión de usuarios entre otros a través de la utilización de interfaces de aplicación (API), proveyéndoles de los elementos básicos que sus aplicativos necesita para funcionar sin tener que construir su propio backend (okta, 2024).

4.10. CONTENEDORES COMO SERVICIO (CAAS)

Los contenedores como servicio o Container as a Service (CaaS) es un modelo de computación en la nube que permite la carga, organización, escalamiento, ejecución y gestión de aplicaciones en contenedores sin la necesidad de gestionar la plataforma o infraestructura en donde se ejecuten estos contenedores (Buchanan, 2025).

4.11. ARQUITECTURAS DISTRIBUIDAS

Modelo de diseño de sistemas en donde múltiples recursos computacionales o nodos trabajan de manera conjunta para ejecutar diferentes actividades para lograr un objetivo común, permitiendo la distribución de tareas y recursos de manera eficiente, mejorando la escalabilidad, la tolerancia a fallos y el rendimiento general de la solución (Zettler, 2025).

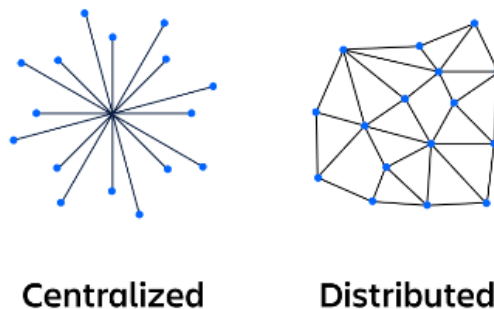


Figura 4. Modelos Centralizados vs Distribuidos. Fuente: (Jung Kang & Jihyoun, 2013)

4.12. ARQUITECTURAS ASÍNCRONAS

Se trata de un enfoque o modelo tecnológico que está pensado para que las operaciones se realicen de manera independiente y no bloqueantes, lo cual permitirá que los diferentes componentes o servicios se ejecuten sin esperar respuestas inmediatas de otros, lo que permite que se puedan ejecutar procesos en paralelo sin esperar que otras operaciones se realicen. Este modelo permite la implementación de arquitecturas distribuidas de alto rendimiento (Cabrera, 2023).

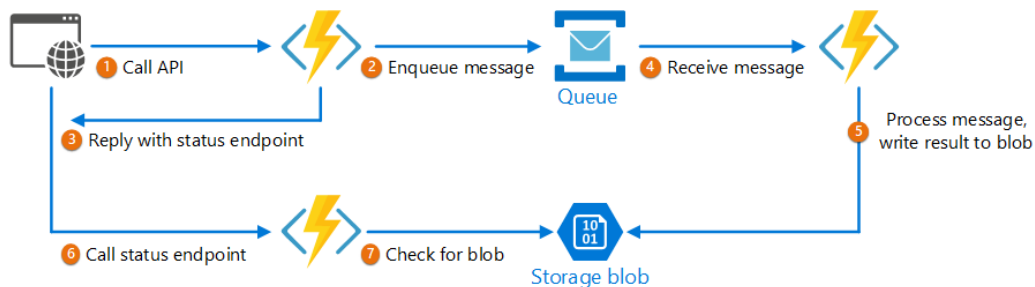


Figura 5. Modelo de referencia de una arquitectura asíncrona. Fuente: (Microsoft Learn, 2025).

De acuerdo con la figura 5, el modelo de arquitectura asíncrono planteado está basado en colas de mensajería o Queues, este modelo es un patrón comúnmente utilizado en plataformas como Azure Functions y está especialmente diseñado para el manejo de altas cargas de trabajo de manera desacoplada. En la figura de izquierda a derecha se puede ver el siguiente flujo:

1. Una aplicación cliente crea una solicitud HTTP a través de la llamada a una API.
2. Luego una función serverless de Azure escribe un mensaje en la cola.
3. Una vez encolado el mensaje la función serverless devuelve una respuesta a la petición inicial, sin procesar de manera directa toda la lógica de negocio de la operación.
4. El mensaje una vez llega a la cola de mensajería una segunda función serverless de Azure se activa de manera automática.

5. Una vez esta función se activa con el mensaje de la Queue inicia la operación que procesa la lógica de negocio y guarda su resultado en un Azure Blob Storage.
6. En una segunda operación desde la aplicación cliente, esta verifica y lee la información en el Azure Blob llamando a una tercera Azure Functions.
7. Si la información se encuentra disponible en el Storage Blob la función serverless responderá con la información disponible, sino indicaría que el proceso sigue en curso lo que evita interbloqueos en una operación que procese una lógica de negocios compleja o con una carga de trabajo intensa.

Entre los beneficios de este tipo de estrategias de arquitecturas asíncronas se puede observar que en un modelo de asincronía completa una aplicación cliente no tiene que esperar a que se procese directamente toda la tarea, con lo que se mejora la experiencia de usuario, además al utilizar una estrategia de colas se puede reintentar procesar nuevamente la operación tantas veces se necesite con lo que se convierte en una solución con una alta tolerancia a fallos.

4.13. HERRAMIENTAS Y TECNOLOGÍAS SERVERLESS

Análisis para la identificación de aquellas herramientas y tecnologías que puedan permitir la implementación de funciones como servicio (Serverless) en IFX, con el objetivo de determinar su comportamiento y características que puedan ser adaptadas a las necesidades del proyecto.

4.13.1. KNATIVE

Knative es una plataforma serverless de código abierto construida sobre Kubernetes que permite ejecutar aplicaciones sin servidor, con soporte basado en eventos. Proporciona una experiencia de desarrollo y despliegue sin servidor (Knative, 2025).

4.13.2. APACHE OPENWHISK

Plataforma de computación sin servidor (serverless) de código abierto que permite ejecutar código en respuesta a eventos sin necesidad de administrar servidores. Se basa en el concepto de "acciones", que son fragmentos de código que se ejecutan cuando se activan por un evento (Apache Software Foundation, 2025).

4.13.3. OPENFAAS

OpenFaaS (Functions as a Service) es un marco de código abierto que permite construir, desplegar y gestionar funciones sin servidor en cualquier infraestructura. Simplifica la implementación de funciones como contenedores con Docker y Kubernetes, lo que facilita su gestión y escalabilidad (OpenFaaS Ltd, 2025).

4.13.4. FN PROJECT

Plataforma de código abierto diseñada para ejecutar funciones sin servidor (serverless) en contenedores Docker, empaquetando y ejecutando funciones, lo que garantiza la portabilidad y la coherencia en diferentes entornos. Fn Project es un motor de eventos de código abierto para la ejecución de funciones (Fn Project, 2025).

5. PROYECTOS DE REFERENCIA

En este apartado se busca identificar las características que se han tenido en cuenta por diferentes investigadores en la implementación de proyectos serverless, buscando realizar un análisis de esos casos de éxito que puedan ser alineados con las necesidades de IFX.

5.1. APLICACIÓN DE TECNOLOGÍAS Y ARQUITECTURAS SERVERLESS PARA EL DESARROLLO IOT

En este proyecto se centra en el diseño de un modelo arquitectónico para aplicar serverless como herramienta o componente para monitorear soluciones de IoT en un Smart home y a partir de sensores controlar el comportamiento lógico de elementos como un calefactor y un climatizador, ajustando el comportamiento de ambos en función de las necesidades de reducción de consumo de energía (Sanfrutos, 2020). El proyecto se enfoca en analizar la viabilidad técnica de utilizar serverless en comparación con otros modelos tradicionales.

- **Fortalezas:** En el proyecto lograron implementar un modelo que valida la capacidad de serverless para IoT en un ambiente controlado, evidenciando que este tipo de arquitecturas ofrecen una mejor administración de recursos de cómputo ya que al evitar el uso de servidores dedicados se reducen costos operativos.
- **Debilidades:** Al utilizar un servicio de AWS el proyecto quedo atado a este proveedor de servicios en la nube, además de no implementar medidas de seguridad robustas en la aplicación del modelo.
- **Similitudes con el proyecto:** A diferencia del objetivo de esta práctica, este proyecto no busca crear un modelo para la implementación de serverless, por el contrario, se limita a utilizar este tipo de tecnologías aplicada sobre un caso en particular.

5.2. ON-PREMISES INFRASTRUCTURE FOR SERVERLESS COMPUTING

Este artículo hace un análisis de los conceptos de computación sin servidor y su aplicación en infraestructuras on-premises, analizando como esta arquitectura podría mejorar la eficiencia para el desarrollo de software (Nagaraj & Sriramulu, 2019). En este se explican conceptos claves como BaaS y FaaS, y mostrando sus diferencias con conceptos como DevOps, PaaS y contenedores, además de explicar cuáles serían las ventajas al reducir costos al simplificar los procesos de desarrollo y brindando una visión de los desafíos que ofrecen los vendor lock-in o dependencias con los proveedores de este tipo de servicios, así como los problemas de latencia en la ejecución de funciones con estrategias pasivas, es decir aquellas que se activan después de recibir un evento.

- **Fortalezas:** El artículo hace un repaso muy completo de los conceptos claves para implementar serverless en ambientes onpremise y brindando una visión general de aquellos casos de uso en donde utilizar serverles, es decir APIs Rest, IoT, chatbots y análisis de datos, además de brindar una visión general para aquellas empresas que busquen no depender de soluciones de nube pública.
- **Debilidades:** Al ser un artículo de Dell technologies se centra en mostrar las ventajas de utilizar sus tecnologías para la adopción de este tipo de estrategias, utilizando por ejemplo Elastic Cloud Storage (ECS) y Pivotal Cloud Foundry (PCF), los cuales son dos de sus productos.
- **Similitudes con el proyecto:** Este artículo está muy relacionado con el objetivo de este proyecto ya que en él se muestran las ventajas y resultados que esperamos obtener con la implementación de este modelo en IFX.

5.3. SERVERLESS COMPUTING, FUNCIONES COMO SERVICIO (FAAS) PARA EL SOPORTE DE CARGAS COMPUTACIONALES EN LA NUBE.

Este proyecto de maestría busca hacer un análisis de las principales plataformas open-source para la implementación de funciones como servicio (FaaS), evaluando las ventajas, desventajas y limitaciones de cada una de ellas a través de pruebas de rendimiento al comparar las distintas estrategias de precarga de funciones que utilizan

estas plataformas open-source. Las plataformas evaluadas en este proyecto fueron Apache OpenWhisk, Fission, Open FaaS y Kubeless (Rodríguez Domínguez, 2020).

- **Fortalezas:** Este proyecto aborda problemas claves que se presentan en la implementación de plataformas FaaS para el uso de funciones como servicio, identificado los puntos fuertes y débiles de cada una de las plataformas evaluadas.
- **Debilidades:** A pesar de mostrar las fortalezas de cada tecnología, la prioridad del ejercicio en este proyecto se centró en la implementación de prototipos, sin embargo, no se evaluaron escenarios reales en donde la implementación de este tipo de plataformas podría funcionar sin problemas, por ejemplo, con desafíos como el monitoreo de los ambientes que permitan identificar consumos desmedidos o su comportamiento al ser integrados con ambientes externos.
- **Similitudes con el proyecto:** Este proyecto brinda una base estable sobre las tecnologías disponibles sobre las que se soportaría este proyecto ya que utilizaría una de estas plataformas para su implementación. Gracias a este proyecto podemos determinar que OpenFaaS se presenta como la mejor opción en cuanto a rendimiento en tiempos de ejecución, escalabilidad, facilidad de uso, comunidad de soporte, soporte a múltiples lenguajes de programación, independencia de entorno de ejecución gracias a su compatibilidad con Kubernetes y alta personalización en la configuración de funciones.

6. METODOLOGÍA

Este trabajo se desarrolla bajo un enfoque aplicado experimental con el propósito de analizar el contexto actual de implementación de servicios de tipo serverless en infraestructura de nube privada, y proponer un modelo arquitectónico asíncrono apoyado en un prototipo que facilite la implementación de servicios de computación sin servidor en la nube privada de IFX Networks.

Se hace uso de la metodología para desarrollo de software cascada como paradigma de desarrollo para la construcción del modelo propuesto, la cual se enfoca en el desarrollo de fases secuenciales iniciando por el análisis de la información actual, posterior, la elaboración del diseño técnico, seguido de la implementación en código fuente y finalmente las validaciones y pruebas del modelo como se muestra en la figura 6.

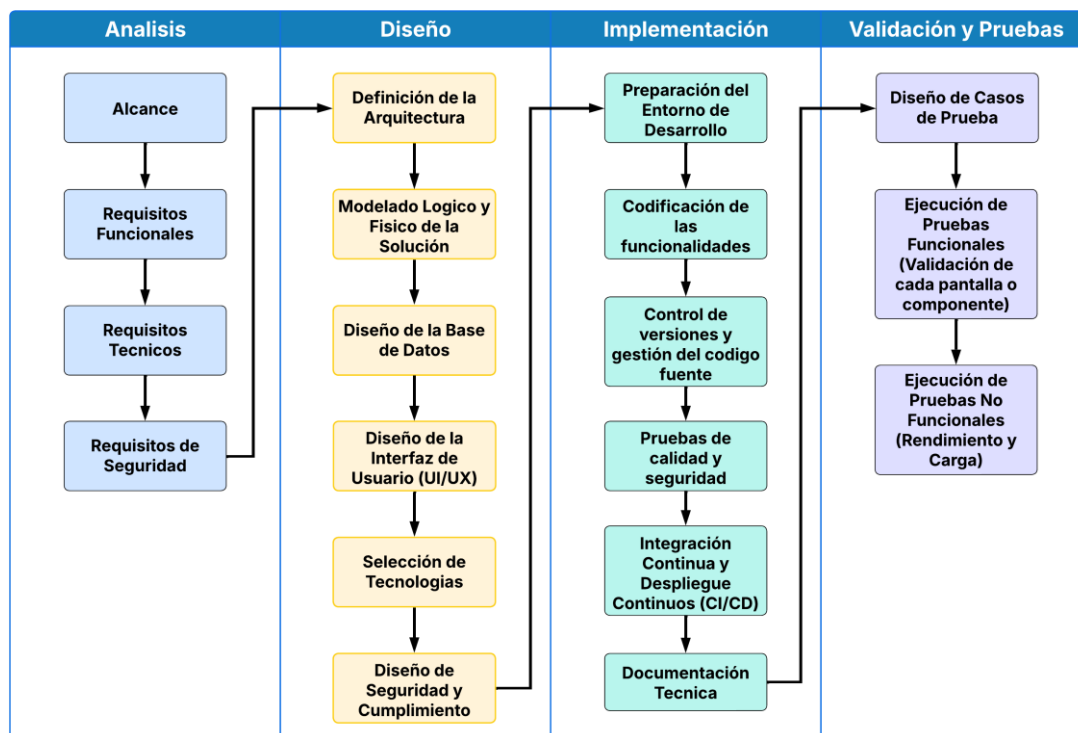


Figura 6. Metodología de desarrollo para la implementación de modelo arquitectónico

Fuente: Elaboración propia

Cada fase ejecuta actividades y crea artefactos técnicos de software requeridos para cumplir con la necesidad planteada para el desarrollo del proyecto.

6.1. FASE DE ANÁLISIS

En esta etapa se realiza el levantamiento de información y el análisis de la misma, constituyendo la base fundamental para el desarrollo del modelo arquitectónico a implementar, en esta fase se recopila de forma estructurada los requerimientos funcionales y técnicos y da solución a la necesidad del negocio y las restricciones del entorno.

Durante esta fase, se identifican los actores involucrados para entender a profundidad los procesos actuales, los marcos de referencia, estándares, patrones arquitectónicos y buenas prácticas para la implementación de servicios en la nube, y se define con claridad qué debe hacer el modelo arquitectónico. Posteriormente, se realiza un análisis detallado de la información obtenida, documentando de manera precisa los requerimientos funcionales, técnicos y de seguridad que requiere el modelo. Esta información es la base y sirve como guía para las siguientes fases, minimizando ambigüedades y facilitando el diseño y la validación del modelo a proponer.

a. Actividades, herramientas y artefactos a implementar en la fase de análisis

- Revisión de literatura
- Determinar el alcance y limitaciones del modelo con base en la identificación de requisitos funcionales, técnicos y de seguridad comunes requeridos para el modelo
- Revisión de guías de seguridad implementados en IFX Networks de organismos como ISO 27001.

b. Entregables

- Listado de requisitos funcionales
- Listado de requisitos técnicos y de seguridad
- Matriz comparativa de modelos existentes y selección de elementos clave para el diseño propuesto.

6.2. FASE DE DISEÑO

El objetivo de esta fase se enfoca en determinar los criterios técnicos, organizacionales y de seguridad que debe cumplir el modelo a proponer para IFX Networks. En el diseño se transforman los requisitos definidos en la fase de análisis en una arquitectura técnica y funcional que guíe la construcción e implementación del modelo arquitectónico asíncrono.

En esta fase se establecen las estructuras del sistema, incluyendo la definición de módulos, interfaces, bases de datos, flujos de información y tecnologías a utilizar. Se elaboran diagramas y especificaciones detalladas que describen cómo se implementarán los requisitos, para el desarrollo del presente proyecto se define el desarrollo de las siguientes actividades y artefactos.

a. Actividades, herramientas y artefactos a implementar en la fase de diseño

- Revisión de documentación técnica de IFX Networks.
- Diagrama de casos de Uso
- Diagrama de contexto
- Diagrama de proceso
- Elaboración de mockup.
- Prototipos
- Diagrama de clases
- Diagrama de despliegue
- Diagrama Modelo entidad relación
- Diagrama de infraestructura
- Diagrama de contenedores
- Determinar herramientas, lenguajes de desarrollo y entorno para el modelo arquitectónico
- Elaboración del modelo arquitectónico de componentes
- Documentar en forma descriptiva las capas, componentes, flujos, y aspectos técnicos del modelo.

b. Entregables

- Diseño del modelo arquitectónico de componentes documentado de acuerdo con todos los diagramas definidos en esta fase.

6.3. FASE DE IMPLEMENTACIÓN

Esta fase abarca la implementación y codificación de las funcionalidades en un prototipo de software sobre la infraestructura tecnológica de IFX Networks con base en los diseños previamente establecidos y utilizando los lenguajes de programación, herramientas y entornos definidos en el diseño técnico.

En esta fase también se realizan pruebas de carga para verificar las funcionalidades desarrolladas.

a. Actividades, herramientas y artefactos a implementar en la fase de implementación:

- Desarrollo del prototipo en la infraestructura privada de IFX Networks sobre los lenguajes de programación establecidos para la validación del modelo propuesto.

b. Entregables

- Código fuente documentado y funcionando en la arquitectura privada de nube de IFX Networks.

6.4. FASE VALIDACIÓN Y PRUEBAS DEL MODELO

Esta fase de la metodología se encarga de verificar que el software desarrollado cumpla con los requisitos definidos en la fase de análisis y que funcione correctamente según lo diseñado. Durante esta etapa se ejecutan las pruebas de carga diseñadas y se obtienen las evidencias de cumplimiento de los requisitos, se incluye la identificación de errores y sus correcciones.

El objetivo es asegurar la calidad, confiabilidad y estabilidad del modelo antes de su salida a producción. Además, se valida que el producto responda a las necesidades del usuario y los objetivos del negocio, garantizando que esté listo para ser entregado y puesto en operación.

a. Actividades, herramientas y artefactos a implementar en la fase de validación y pruebas

- Creación y ejecución de pruebas técnicas de carga por medio de script técnicos en herramienta jmeter.
- Ejecución de pruebas de carga.

b. Entregables

- Evidencia de ejecución de pruebas
- Informe de resultados de pruebas

7. DESARROLLO DE LA PROPUESTA

En este capítulo se detallan los resultados del desarrollo del modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de la compañía IFX Networks, el cual fue construido siguiendo la metodología propuesta. Esta metodología permitió abordar el proceso de desarrollo de forma ordenada y secuencial, asegurando que cada fase se completara antes de iniciar la siguiente.

Durante la etapa de análisis, se recopilaron los requerimientos funcionales, técnicos y de seguridad mediante la revisión documental, y conocimiento de la infraestructura actual. En la fase de diseño, se elaboraron diferentes vistas en diagramas que detallan capas, componentes, flujos y consideraciones técnicas de acuerdo con el alcance de cada diagrama. Se presenta el diagrama de casos de uso, diagrama de contexto, mockup, prototipos, diagrama de clases, diagrama de despliegue, diagrama de proceso BPMN, diagrama modelo entidad relación, diagrama de infraestructura, diagrama de contenedores, diagrama de implementación, y finalmente el modelo arquitectónico de componentes.

Posterior, en la etapa de implementación se prototipa el modelo en los lenguajes de programación definidos, aplicando buenas prácticas de programación, y documentando el código fuente para facilitar su mantenimiento en futuras mejoras e implementaciones.

Durante la etapa de pruebas y validación se realizaron pruebas de carga que permitieron verificar el correcto funcionamiento del modelo. Los resultados evidenciaron que el modelo responde adecuadamente a los requisitos planteados, mostrando un desempeño estable y cumpliendo con las funcionalidades definidas.

En los siguientes apartados se presentan los resultados técnicos y los artefactos obtenidos en cada una de las etapas implementadas.

7.1. FASE DE ANÁLISIS

A continuación, se relaciona cada uno de los artefactos y resultados de esta fase, se listan los requisitos funcionales, técnicos y de seguridad y la matriz comparativa de los servicios actuales en el mercado.

Para la actividad revisión de documentación se registra en los capítulos: 4. Marcos de Referencia, 5. Proyectos de Referencia, y 6. Herramientas y Tecnologías Serverless del presente documento.

7.1.1. REQUISITOS FUNCIONALES

La tabla 1, registra los requisitos funcionales numerados con la notación RQF y el número de Requisito.

RQF: Requisito Funcional

ID REQ	Descripción del Requisito
RQF-001	El modelo debe presentar una plataforma web que permita a los desarrolladores de IFX Networks crear y administrar funciones como código de los proyectos de los equipos de desarrollo que hacen parte de la fábrica de software de IFX Networks.
RQF-002	El modelo debe presentar opciones de creación para cada uno de los siguientes equipos y desarrolladores de la fábrica de software de IFX Networks: devcloud, devdigital, devnetwork, devops, devnetsuite. Cada uno de los cuales es encargado de una línea de negocio de IFX Networks, y tienen infraestructura dedicada para cada una de las automatizaciones de sus equipos.
RQF-003	El modelo debe mostrar una pantalla interfaz que permita crear funciones a partir de los lenguajes de programación node.js, Python, y Go.
RQF-004	El modelo debe permitir la gestión y uso de variables seguras dentro de las funciones.

ID REQ	Descripción del Requisito
RQF-005	La plataforma debe generar código fuente a partir del asistente de Inteligencia Artificial utilizando los modelos de IFX Networks.
RQF-006	El modelo debe permitir validar el código a partir de modelos de inteligencia artificial de IFX Networks.
RQF-007	Se debe poder utilizar funciones de código genéricos asociados a los diferentes lenguajes dispuestos.
RQF-008	Se debe poder desplegar en la infraestructura de kubernetes de los equipos de desarrollo de IFX Networks.
RQF-009	El modelo debe permitir el despliegue de funciones serverless en la infraestructura de nube privada de IFX Networks.

Tabla 1. Listado de requisitos funcionales. Fuente: Elaboración propia

7.1.2. REQUISITOS TÉCNICOS Y DE SEGURIDAD

La tabla 2 registra los requisitos técnicos y de seguridad numerados con la notación RQS y el número de Requisito.

RQS: Requisito técnico y de seguridad.

ID REQ	Descripción del Requisito
RQS-001	El despliegue de funciones debe hacerse de manera asíncrona evitando conflictos al momento del despliegue de las funciones.
RQS-002	El acceso a la plataforma de funciones estará disponible a través de VPN.
RQS-003	Las capas de presentación, de implementación y despliegue deben estar controladas a través de firewall físico y web application firewall (WAF).
RQS-004	El tiempo de respuesta de las funciones debe ser menor a 30 segundos.
RQS-005	La arquitectura debe ser modular y escalable.
RQS-006	Los lenguajes de desarrollo deben estar dentro de los lenguajes aprobados por los lineamientos y políticas de la fábrica de software de IFX Networks.
RQS-007	Compatibilidad multiplataforma con soporte para navegadores de última generación.

ID REQ	Descripción del Requisito
RQS-008	Las funciones desplegadas deben estar contenerizadas y respaldadas en un registry Docker.
RQS-009	Se debe implementar con gestión de errores incluyendo mecanismos de manejo de excepciones y registro de errores (logs).
RQS-010	Debe tener control de versiones y uso de sistemas como Git para gestión del código fuente.
RQS-011	El modelo debe contar con documentación técnica: Manuales, diagramas y especificaciones claras del funcionamiento del sistema y su código.

Tabla 2. Requisitos técnicos y de seguridad. Fuente: Elaboración propia

7.1.3. MATRIZ COMPARATIVA DE MODELOS EXISTENTES Y SELECCIÓN DE ELEMENTOS CLAVE PARA EL MODELO PROPUESTO

Para el desarrollo del modelo arquitectónico basado en arquitectura Serveless, se analizaron los diversos proveedores de nube que ofrecen servicios Functions-as-a-Services (FaaS), y se analizaron las diferentes características con el fin de evaluar la pertinencia del modelo propio para IFX Networks.

A continuación, se presenta una matriz comparativa de las principales características, ventajas y desventajas de las opciones del mercado como son: AWS Lambda (Lambda, 2025), Azure Functions (Functions, 2025), Google Cloud Functions (Cloud, 2025), y se incluye la que será el modelo de IFX FaaS, con el fin de identificar los requerimientos técnicos y de infraestructura del nuevo modelo para IFX. Adicional, la matriz resalta las características modelo de pago y control y portabilidad como características diferenciales y fundamentales para la implementación del nuevo modelo arquitectónico asíncrono para servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks.

Característica	AWS Lambda (Lambda, 2025)	Azure Functions (Functions, 2025)	Google Cloud Functions (Cloud, 2025)	IFX FaaS
Proveedor	Amazon Web Services	Microsoft Azure	Google Cloud Platform	IFX Networks
Modelo de ejecución	Basado en eventos / completamente serverless	Basado en eventos / serverless	Basado en eventos / serverless	Contenedores (Docker) / autoalojado
Lenguajes soportados	Node.js, Python, Java, Go, .NET, Ruby, etc.	C#, JavaScript, Python, Java, PowerShell	Node.js, Python, Go, Java, Ruby, PHP	Node.js, Python, Go.
Modelo de despliegue	Consola, CLI, SDK, SAM	Portal, CLI, Visual Studio, GitHub	GCP Console, CLI, Cloud Build	Plataforma WEB propia (Minos)
Escalabilidad automática	Sí	Sí	Sí	Si (a demanda)
Modelo de pago	Por ejecución y tiempo consumido	Por consumo o plan dedicado	Por uso (invocaciones y duración)	Infraestructura propia (costo autogestionado)
Integración con servicios nativos	Alta (S3, DynamoDB, SNS, etc.)	Alta (Blob, Event Grid, Cosmos DB, etc.)	Alta (Pub/Sub, Firestore, etc.)	Limitada (requiere configuración manual)
Control y portabilidad	Limitado (vendor lock-in)	Limitado	Limitado	Alto (open source, portable vía Docker)
Seguridad y autenticación	IAM, VPC, KMS	Azure AD, RBAC, Managed Identity	IAM, VPC, Secret Manager	Según configuración (Kubernetes RBAC, OIDC)}
Comunidad y madurez	Muy alta y madura	Alta y madura	Alta	Comunidad propia -- desarrollo cerrado

Tabla 3. Matriz comparativa servicios FaaS Fuente: Elaboración propia con base en las fuentes consultadas.

7.2. FASE DE DISEÑO

En esta fase se realizan los modelos base donde se ejemplifican los componentes que hacen parte de la arquitectura del modelo a implementar. Los artefactos diseñados para la obtención del modelo arquitectónico son: el diagrama de Casos de Uso, el diagrama de contexto, el diagrama de proceso, el *mockup*, los prototipos, y todos los siguientes modelos que definen las capas, los componentes y las consideraciones técnicas de

acuerdo con cada vista que representa en cada uno, como son: el diagrama de clases, el diagrama de despliegue, el diagrama Modelo Entidad Relación, el diagrama de infraestructura, el diagrama de contenedores, y por último el diagrama del modelo arquitectónico de componentes propuesto.

En este apartado también se define las herramientas, lenguajes de desarrollo y entorno para el modelo arquitectónico propuesto. A continuación, se registra cada uno de los artefactos y documentación relacionada para esta fase de diseño.

7.2.1. DOCUMENTACIÓN TÉCNICA DE IFX NETWORKS

En la fase de Diseño se hace necesario conocer la documentación técnica de IFX Networks que aplica para el abordaje del modelo arquitectónico a saber:

- Política de desarrollo seguro en el Centro de Desarrollos y Automatizaciones de IFX (Devcenter). Documento: DES-PL07 de Gestión de sistema de información de 2024.

Esta política busca formalizar el compromiso de la dirección de IFX a través de políticas, principios, y reglas, que protejan y guíen el desarrollo de aplicaciones en IFX bajo estándares de calidad y seguridad, para la fábrica del centro de desarrollos y automatizaciones (DevCenter). (IFX Networks, 2024).

- Estándar codificación segura en IFX. Documento: DES-ES06 de Gestión de sistema de información de 2024.

El objetivo de este estándar es asegurar que todos los equipos de desarrollo que hacen parte del Centro de desarrollo y Automatización de IFX, adopten y cumplan con los estándares de codificación segura establecidos para todas las aplicaciones y servicios desarrollados. (IFX Networks, 2024).

- Política control de cambios en el código de desarrollo DES-PL06 - Gestión de Sistemas de Información 2024.

El objetivo de este documento es establecer un marco de políticas y procedimientos para la gestión de cambios de los proyectos de desarrollo dentro de nuestra organización. Se busca garantizar que todos los cambios en el código fuente sean gestionados de manera eficiente y segura, minimizando riesgos y asegurando la calidad del software. (IFX Networks, 2024).

7.2.2. DIAGRAMA DE CASOS DE USO

El diagrama de caso de uso captura el comportamiento de un sistema, subsistema o de una clase, reparte la funcionalidad del sistema en transacciones significativas para los actores-usuarios ideales de un sistema (Rumbaugh, Jacobson, & Booch, 2000). La figura 7 muestra el diagrama de casos de uso para el modelo arquitectónico propuesto, mostrando los actores, los usos y las relaciones definidos para la funcionalidad del modelo.

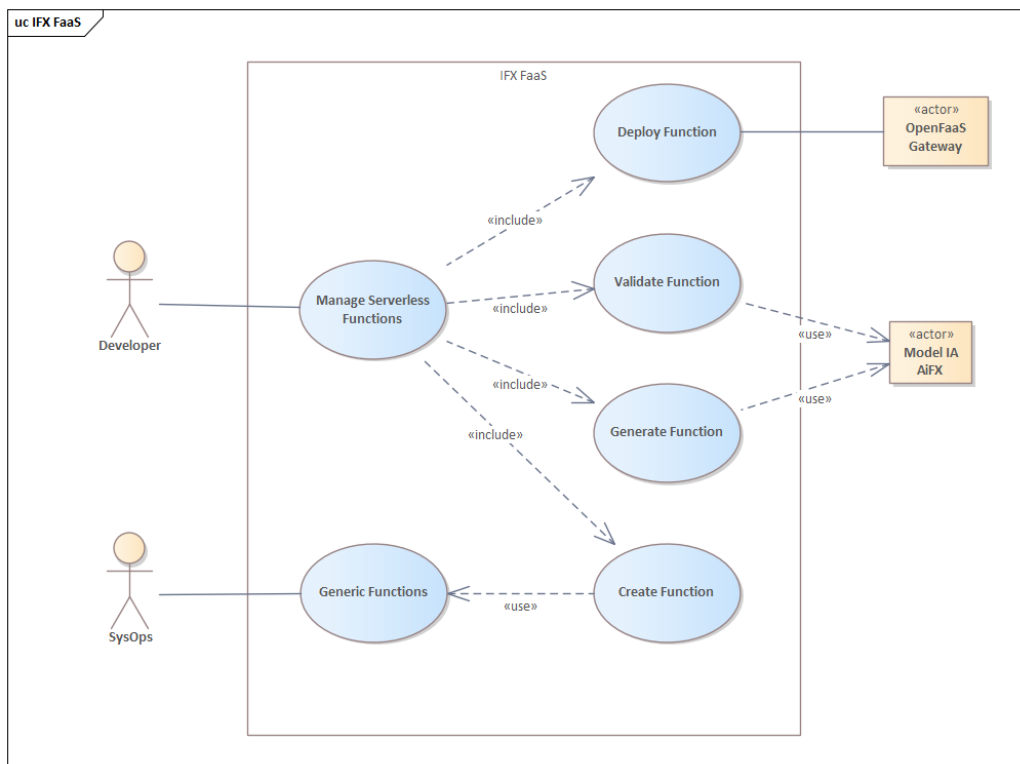


Figura 7. Diagrama de caso de Uso. Fuente: Elaboración propia

7.2.3. DIAGRAMA DE CONTEXTO

El diagrama de contexto presenta el sistema como una sola entidad mostrando las entidades externas y los flujos de información o interacción identificando claramente las interacciones externas (TI, 2018). Para el modelo arquitectónico el diagrama de contexto está compuesto por las entidades Funtions Database entidad que representa la base de

datos para el almacenaje del código fuente de las funciones, Secrets para las variables secretas utilizadas en las funciones, OpenFaaS herramienta donde se despliega las funciones como servicio, y finalmente la entidad Snippets, para fragmentos de código comunes para las funciones como se muestra en la figura 8.

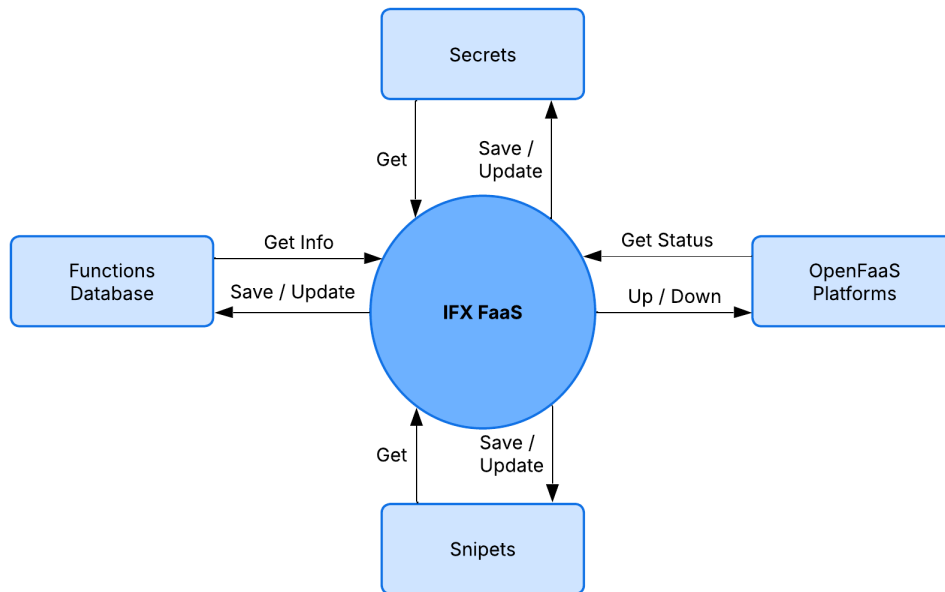


Figura 8. Diagrama de contexto. Fuente: Elaboración propia

7.2.4. DIAGRAMA DE PROCESO

Un diagrama de procesos BPMN (Business Process Model and Notation) permite aplicar una notación gráfica estandarizada legible y entendible para modelar los procesos del negocio de manera comprensible para todos los interesados. Este diagrama utiliza la notación BPMN para describir el flujo de actividades, los eventos, y las decisiones dentro del proceso que se está representando (Maquera Quispe, Ticse Capcha, Gómez Morales, & Quintana, 2019). La figura 9, detalla el proceso en notación BPMN del modelo arquitectónico propuesto. Su lectura gráfica presenta el flujo y las actividades para cada uno de los involucrados.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

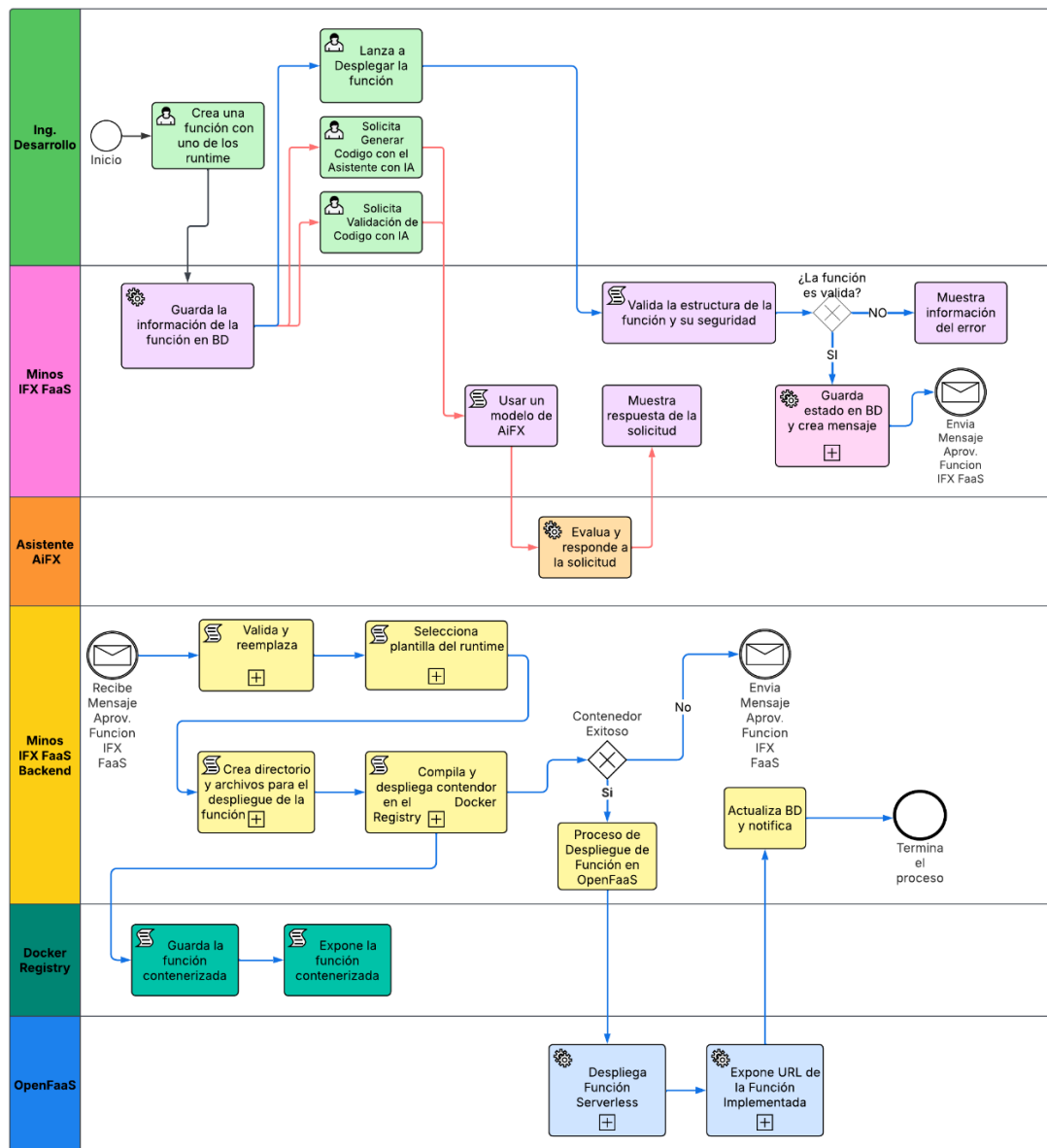


Figura 9. Diagrama de Proceso BPMN. Fuente: Elaboración propia

7.2.5. MOCKUP

El siguiente mockup mostrado en la figura 10 muestra la representación gráfica y visual del componente para la edición de código fuente de las funciones Serverless del proyecto, el cual consta de un título con el nombre de la función, la url de la función desplegada, el editor de código fuente, y los botones para el guardado y despliegue de la función.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks



Figura 10. Mockup de la plataforma. Fuente: Elaboración propia

7.2.6. PROTOTIPO PLATAFORMA WEB

A continuación, se presentan los prototipos diseñados para la creación de funciones por cada uno de los grupos de desarrolladores de la fábrica de software de IFX Networks.



Figura 11. Prototipo plataforma WEB. Fuente: Elaboración propia

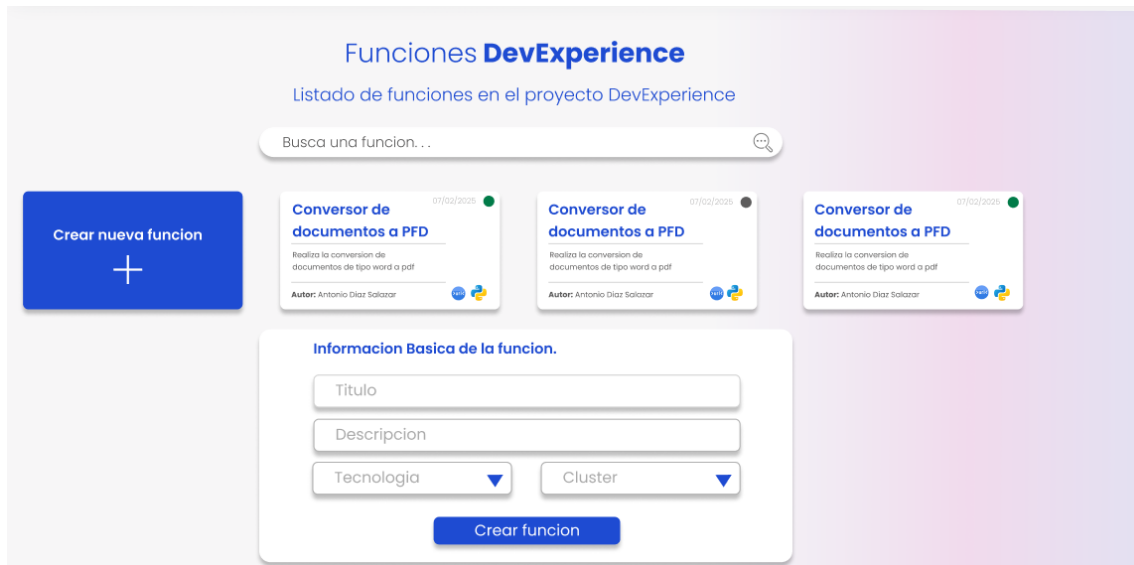


Figura 12. Prototipo Crear función. Fuente: Elaboración propia

7.2.7. DISEÑO DE LA ARQUITECTURA

El modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks, en adelante nombrado como **IFX FaaS**, está conformado por la combinación de diferentes patrones de arquitecturas modernas como Micro Frontend combinado con control de acceso basado en roles (RBAC), API Gateway para el acceso y control de las APIs de la solución, y estrategias de mensajería y streaming (AMQP) para la comunicación asíncrona y la gestión de procesos pesados en la solución. Estos artefactos técnicos y su distribución son descritos a continuación, los cuales son presentados en las siguientes vistas que detallan las capas, los componentes, los flujos, y todas las consideraciones técnicas necesarias para la implementación del modelo **IFX FaaS**.

De esta forma, los siguientes artefactos hacen parte integral del modelo asíncrono IFX FaaS desde cada una de las vistas que representan: Diagramas de clases, Diagrama de despliegue, Modelo Entidad Relación, Diagrama de infraestructura, Diagrama de contenedores, Diagrama de implementación, y finalmente el diagrama con la vista arquitectónica del modelo.

7.2.7.1. DIAGRAMA DE CLASES DE IFX FAAS

A continuación, se presentan dos ilustraciones con el diagrama clases, figura 13 y 14, y posterior se detalla la descripción de los componentes del diagrama, y los patrones de diseño implementados en la tabla 4 y tabla 5 respectivamente.

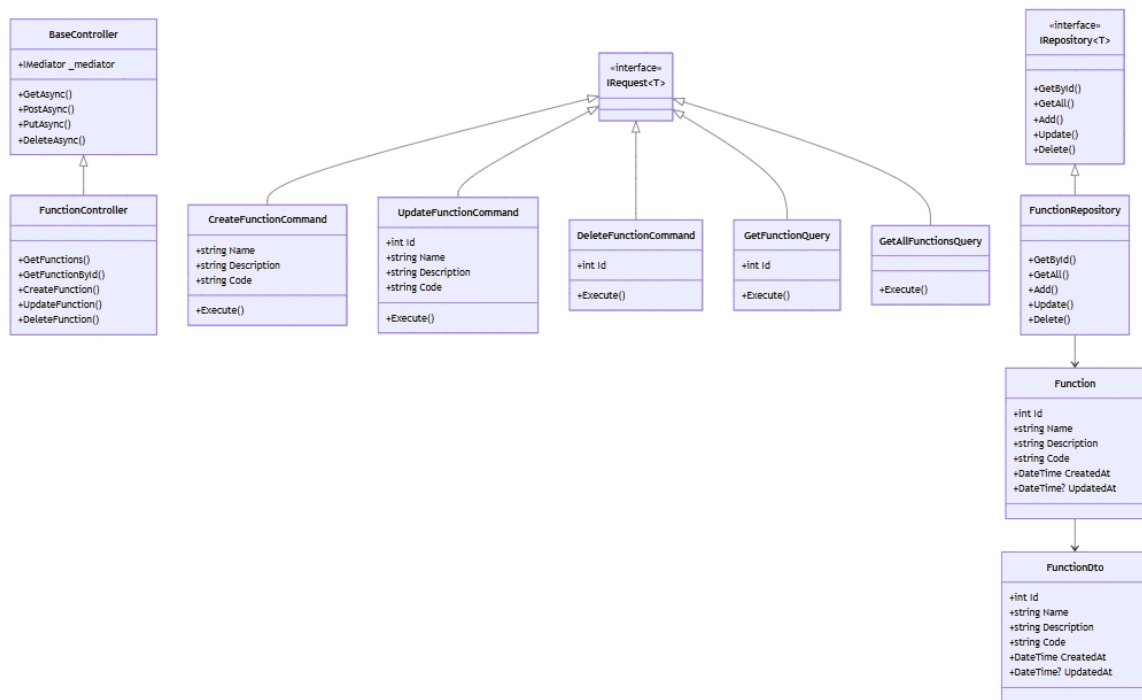


Figura 13. Diagrama de clases de IFX FaaS parte 1. Fuente: Elaboración propia

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

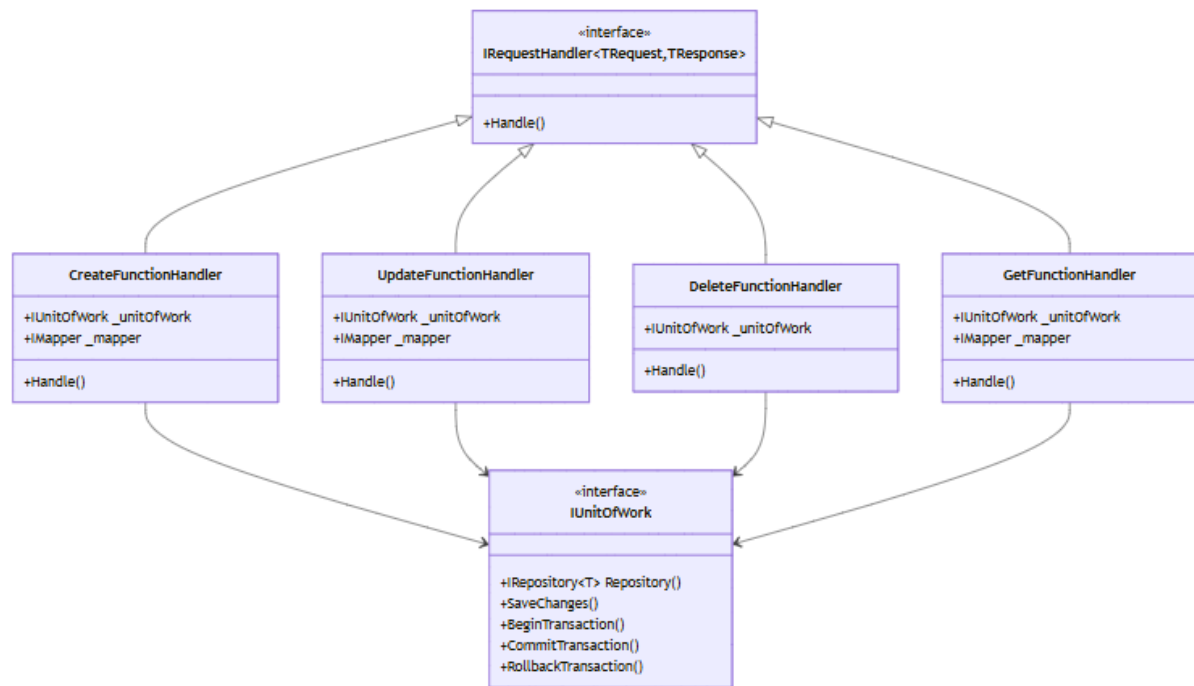


Figura 14. Diagrama de clases de IFX FaaS parte 2. Fuente: Elaboración propia

Descripción de Componentes

Componente	Clase	Descripción para el modelo IFX FaaS
Controllers	BaseController	Controlador base que proporciona funcionalidad común para todos los controladores que exponen las funcionalidades de los métodos de IFX FaaS.
	FunctionController	Controlador específico para el manejo de las operaciones CRUD de IFX FaaS.
Commands	CreateFunctionCommand:	Comando para crear una nueva función.
	UpdateFunctionCommand:	Comando para actualizar una función existente.
	DeleteFunctionCommand	Comando para eliminar una función
Queries	GetFunctionQuery	Consulta para obtener una función específica
	GetAllFunctionsQuery	Consulta para obtener todas las funciones
DTOs	FunctionDto	Objeto de transferencia de datos para funciones
Entities	Function	Entidad principal que representa una función en IFX FaaS

Componente	Clase	Descripción para el modelo IFX FaaS
Repositories	IRepository	Interfaz genérica para operaciones de repositorio
	FunctionRepository	Clase repositorio para la implementación las funcionalidades específicas para los métodos de las operaciones de IFX FaaS.
Handlers	CreateFunctionHandler:	Manejador para crear funciones.
	UpdateFunctionHandler	Manejador para actualizar funciones
	DeleteFunctionHandler	Manejador para eliminar funciones
	GetFunctionHandler	Manejador para obtener funciones
Unit of Work	IUnitOfWork	Interfaz que maneja las transacciones y el contexto de la base de datos

Tabla 4. Descripción de componentes modelo IFX FaaS. Fuente: Elaboración propia

Patrones de Diseño Implementados

Patrón de Diseño	Descripción para el modelo IFX FaaS
CQRS (Command Query Responsibility Segregation) Repository Pattern	- Sirve para realizar una separación clara entre comandos (modificaciones) y consultas (lecturas). - Implementado a través de Commands y Queries - Abstracción de la capa de acceso a datos - Implementación genérica con IRepository
Unit of Work	- Manejo de transacciones y contexto de base de datos - Garantiza la consistencia de los datos
Mediator Pattern	- Desacoplamiento entre componentes - Implementado a través de IMediator
DTO Pattern	- Transferencia de datos entre capas - Prevención de exposición de entidades internas

Tabla 5. Patrones de diseño implementados modelo IFX FaaS. Fuente: Elaboración propia

7.2.7.2. DIAGRAMA DE DESPLIEGUE DE IFX FAAS

Las figuras 15 y 16 representan el diagrama de despliegue del modelo IFX FaaS mostrando la arquitectura física del sistema y los componentes de software que se asocian a los nodos de hardware y su interconexión.

De izquierda a derecha se observa que el acceso se hace directamente a través del navegador comunicándose con la capa de presentación desplegada en el cluster de

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

Kubernetes Devcloud, en este cluster se encuentra la arquitectura de Micro Frontend compuesta por una aplicación en React que se comunica con el Api Gateway, ambos contenerizados en Docker, posterior, el Api Gateway se comunica con la aplicación desarrollada en ASP Net Core, contenerizada y desplegada en el *cluster* de DevDigital, comunicándose con la capa DMZ donde se encuentra alojada la base de datos y el bróker de mensajería (AMQP).

Finalmente, el diagrama de despliegue muestra que las funciones como servicio estarán desplegadas en los diferentes *cluster* de Kubernetes del centro de desarrollo de IFX representados en la última capa.

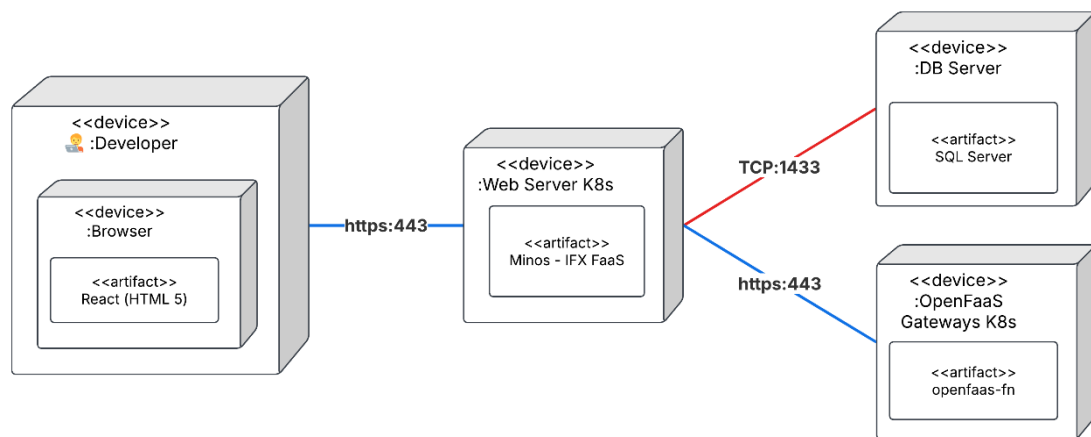


Figura 15. Diagrama de despliegue genérico Modelo IFX FaaS. Fuente: Elaboración propia

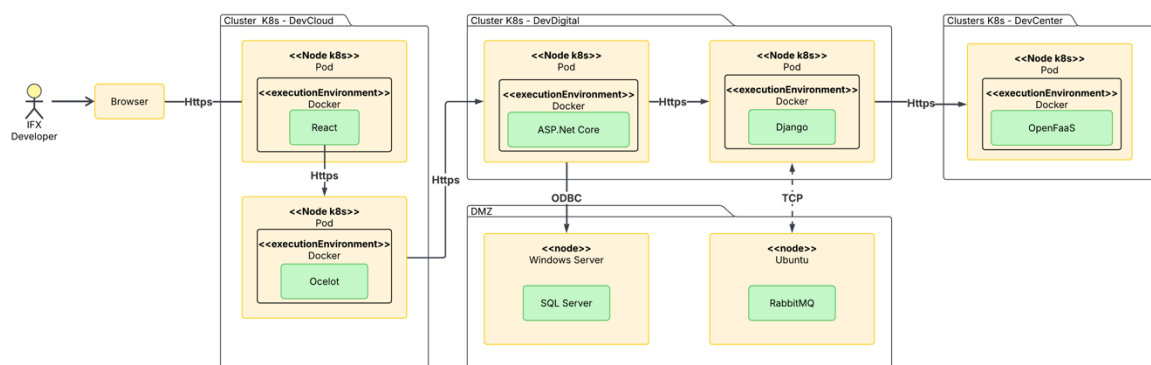


Figura 16. Diagrama de despliegue detallado modelo IFX FaaS. Fuente: Elaboración propia

7.2.7.3. DIAGRAMA MODELO ENTIDAD RELACIÓN MODELO IFX FAAS

Un modelo de datos relacional es la representación gráfica de las entidades, atributos, relaciones, llaves primarias, llaves foráneas y restricciones a nivel de base datos. (ELMASRI & NAVATHE, 2007). La figura 17 presenta el modelo relacional detallando las entidades requeridas y sus relaciones.

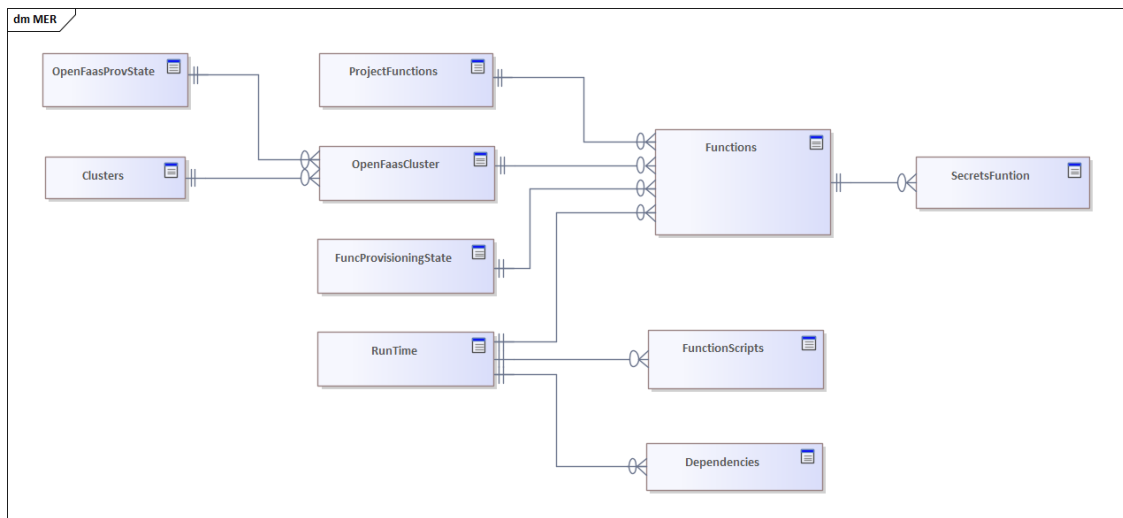


Figura 17. Diagrama MER Modelo Entidad Relación modelo IFX FaaS. Fuente: Elaboración propia

7.2.7.4. DIAGRAMA DE INFRAESTRUCTURA MODELO IFX FAAS

El diagrama de infraestructura es presentado en la figura 18 detallando de izquierda a derecha que el acceso a la aplicación y comunicación se hace únicamente por vía VPN pasando por un Firewall físico y un Web Application Firewall (WAF) individual con el fin de mantener la seguridad en la capa de presentación, al igual que los componentes de Frontend y Backend. Para la comunicación entre el backend y los OpenFaaS Gateways se replica la misma estrategia de tráfico de red usando un firewall físico y WAF.

En el primer grupo se presenta el clúster de Kubernetes de DevCloud y sus nodos, el cual está destinado al despliegue de aplicaciones de capa de presentación en IFX, en el segundo grupo se detalla los nodos del clúster de Kubernetes de DevDigital destinado para el despliegue de aplicaciones operacionales. Este cluster que se

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

encuentra aislado en una zona desmilitarizada o DMZ las aplicaciones automatizan los procesos que tienen comunicación con las plataformas de control de IFX, donde finalmente operarán las Funciones OpenFaaS aisladas.

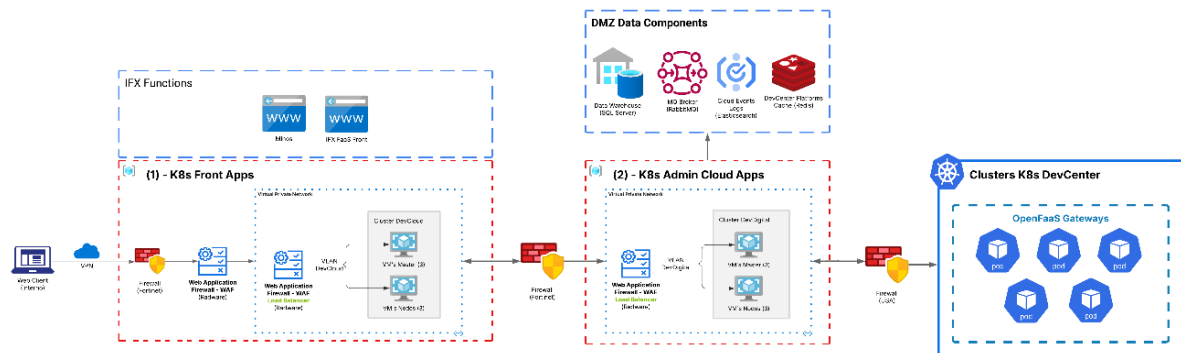


Figura 18. Diagrama de Infraestructura Modelo IFX FaaS. Fuente: Elaboración propia

7.2.7.5. DIAGRAMA DE CONTENEDORES MODELO IFX FAAS

La implementación de las funciones como servicios en este modelo siguen la estrategia de despliegue de funciones contenerizadas con Docker y desplegadas en Kubernetes a través de OpenFaaS, estos clústeres son aquellos que sirven para operación y la automatización de procesos en IFX, cada uno de ellos cuentan con 6 nodos o máquinas virtuales para el balanceo de carga de las aplicaciones desplegadas en ellos, como se presenta en el diagrama de contenedores de la figura 19.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

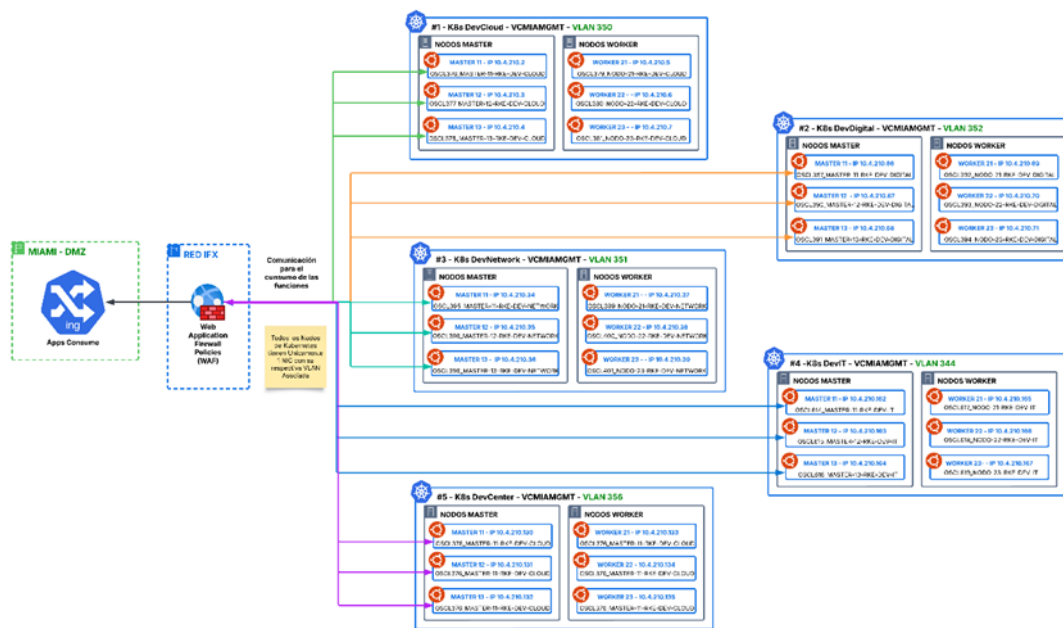


Figura 19. Diagrama de contenedores Modelo IFX FaaS. Fuente: Elaboración propia

7.2.7.6. VISTA ARQUITECTÓNICA DEL MODELO IFX FAAS

La figura 20 detalla el comportamiento del modelo IFX FaaS y proporciona la base sobre la cual se realiza la implementación. En primer lugar, de izquierda a derecha, el diagrama presenta los componentes de la capa de presentación utilizando la herramienta Minos con modelo de Micro Frontend desde la cual se implementa el Micro Frontend “IFX Functions Frontend”, y el API Gateway dedicado a orquestar las APIs que se conectan con los diferentes componentes del frontend de Minos.

Posterior, el modelo presenta la aplicación “Kube Provisioning” (componente diseñado bajo el modelo arquitectónico Clean Code) dedicada a la gestión de las automatizaciones de la infraestructura cloud de IFX Networks. Este componente se comunica con el bus de mensajería para la gestión de procesos asíncronos.

El bus tiene interacción con el componente IFX function backend, componente dedicado a la comunicación con las diferentes plataformas desplegadas en las áreas de alta seguridad HSA de la infraestructura cloud de IFX Networks, y sobre las cuales están implementadas las aplicaciones Open FaaS para la ejecución de las funciones como servicios contenerizadas.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

7.3.1.1. Ventana con el listado de proyectos *serverless* para los equipos del centro de desarrollo de IFX

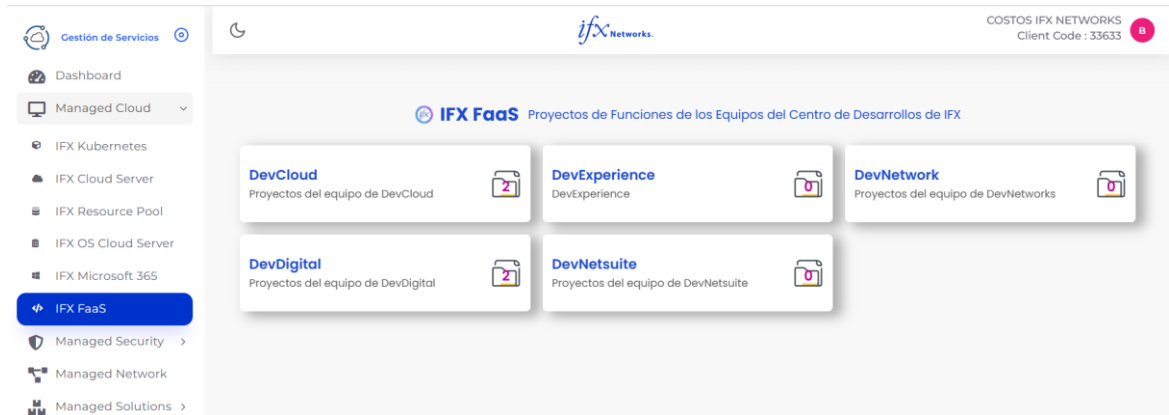


Figura 21: Ventana principal IFX FaaS. Fuente: Elaboración Propia

7.3.1.2. Creación de funciones

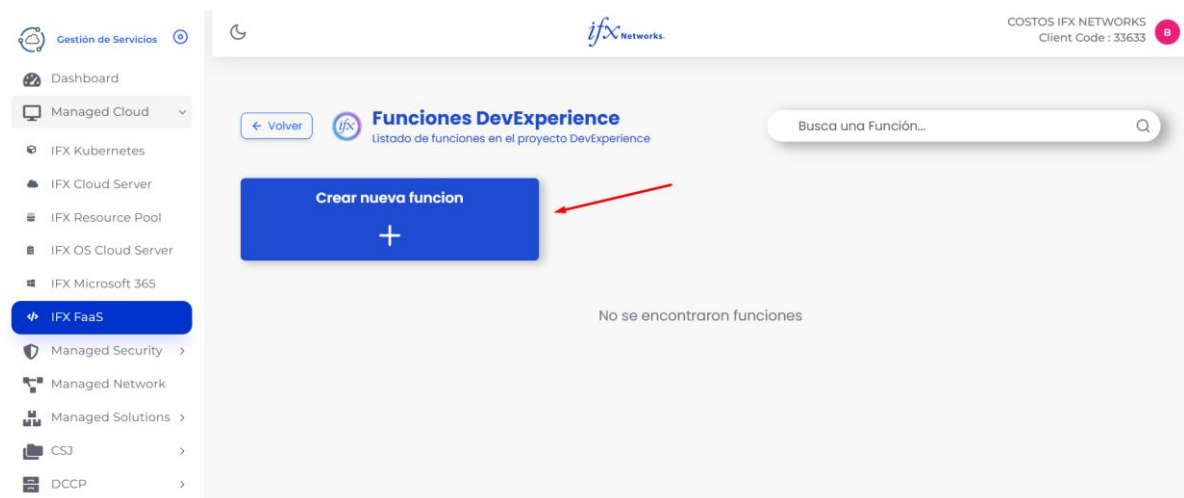


Figura 22: Modulo con el listado de funciones de un proyecto. Fuente: Elaboración propia.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

The screenshot shows the IFX Networks 'Funciones DevExperience' interface. On the left is a sidebar with navigation options: Gestión de Servicios, Dashboard, Managed Cloud, IFX Kubernetes, IFX Cloud Server, IFX Resource Pool, IFX OS Cloud Server, IFX Microsoft 365, IFX FaaS (highlighted), Managed Security, Managed Network, Managed Solutions, CSJ, and DCCP. The main panel displays the 'Funciones DevExperience' header with a 'Volver' button and a list of functions. A modal window titled 'Información básica de la función.' is open, containing the following fields:

- Título:** Multiplicador de matrices
- Descripción:** Esta es una función que sirve para calcular ecuaciones cuadráticas
- Tecnología:** Node 20 (selected from a dropdown)
- Ambiente:** K8s Test (selected from a dropdown)
- Crear Función:** A blue button to submit the form.

Figura 23: Formulario para la creación de funciones. Fuente: Elaboración propia.

The screenshot shows the IFX Networks 'Función: Multiplicador de matrices' editor interface. The sidebar is identical to the previous figure. The main panel shows the function name 'Función: Multiplicador de matrices' with a 'Volver' button and a status message 'Tu app aún no ha sido desplegada'. Below this is a code editor with the following code:

```
1
2 "use strict"
3 module.exports = async (context, callback) =>
4 {
5   return {status: "done"}
6 }
```

At the top right of the code editor are buttons for 'Testing', 'Deploy', and other icons. At the bottom of the editor is a prompt: 'Genera tu funcion con ayuda de AIIX, que deseas generar?' with a search icon.

Figura 24: Editor de código de funciones. Fuente: Elaboración propia.

7.3.1.3. Asistente para generar código a partir de un asistente con IA

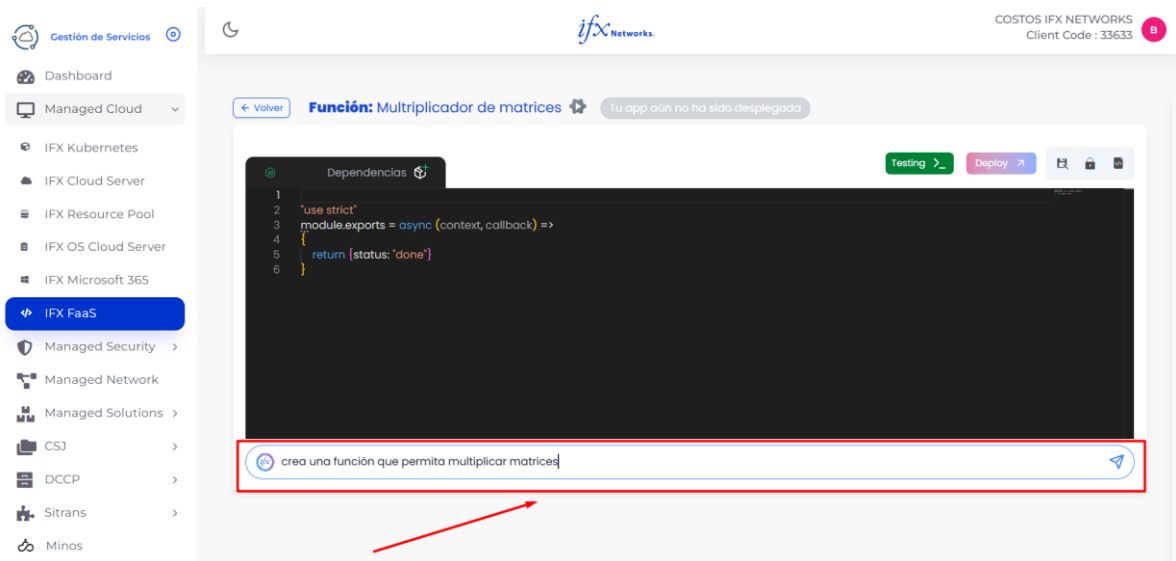


Figura 25: Asistente con IA para la generación de código. Fuente: Elaboración propia.

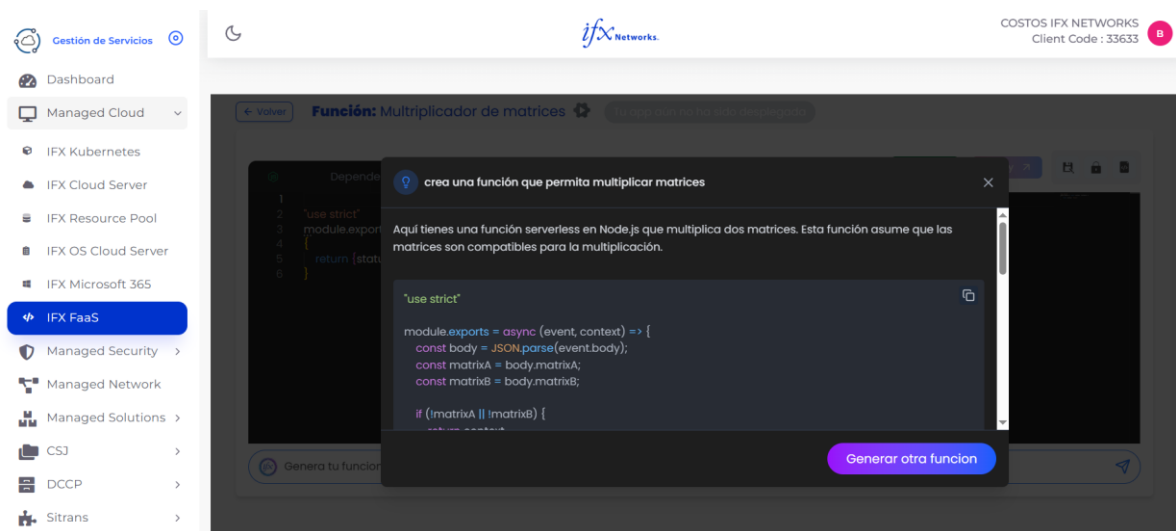


Figura 26: Respuesta del asistente con IA para la generación de código. Fuente: Elaboración propia

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

a. Validación o testing de las funciones antes de ser desplegadas

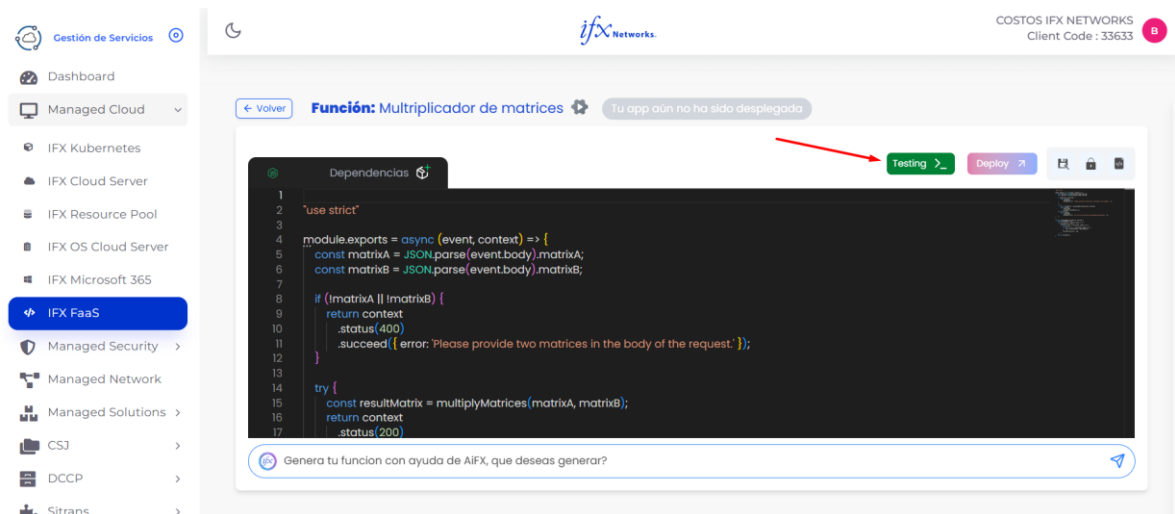


Figura 27: Validación del código con IA. Fuente: Elaboración propia

b. Resultado sin errores:

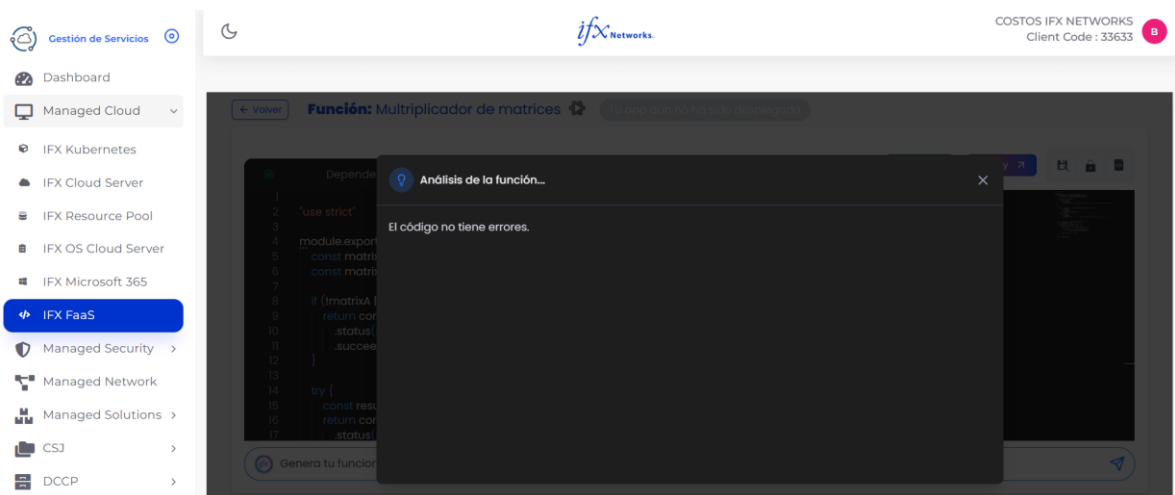


Figura 28: Resultado de asistente que valida código cuando no hay errores. Fuente: Elaboración propia.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

- c. Resultado con errores en el código: Se elimina este fragmento de código para la validación

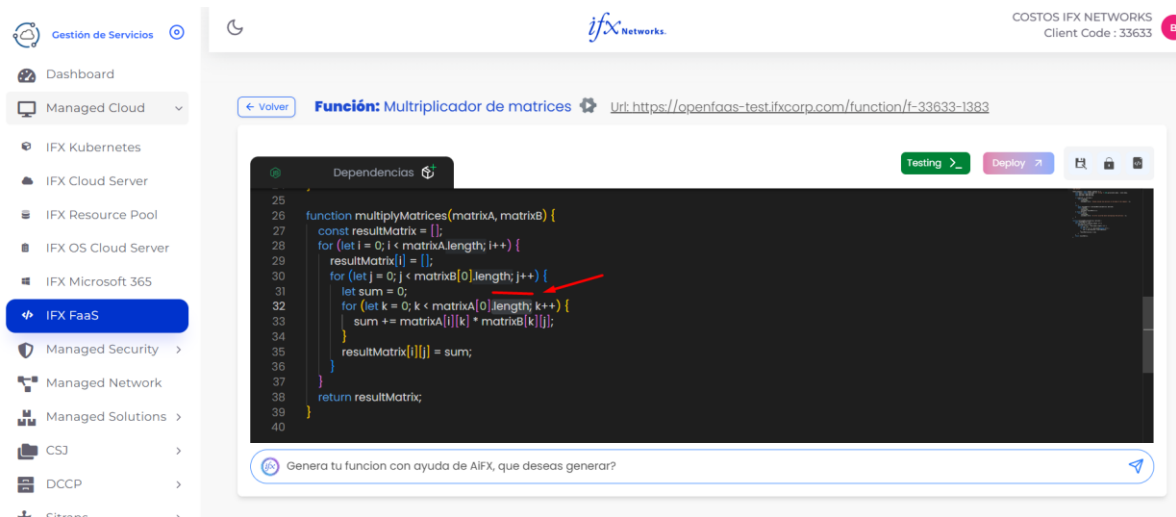


Figura 29: Respuesta a escenarios con errores - 1. Fuente: Elaboración propia.

- d. Procesa la validación del código

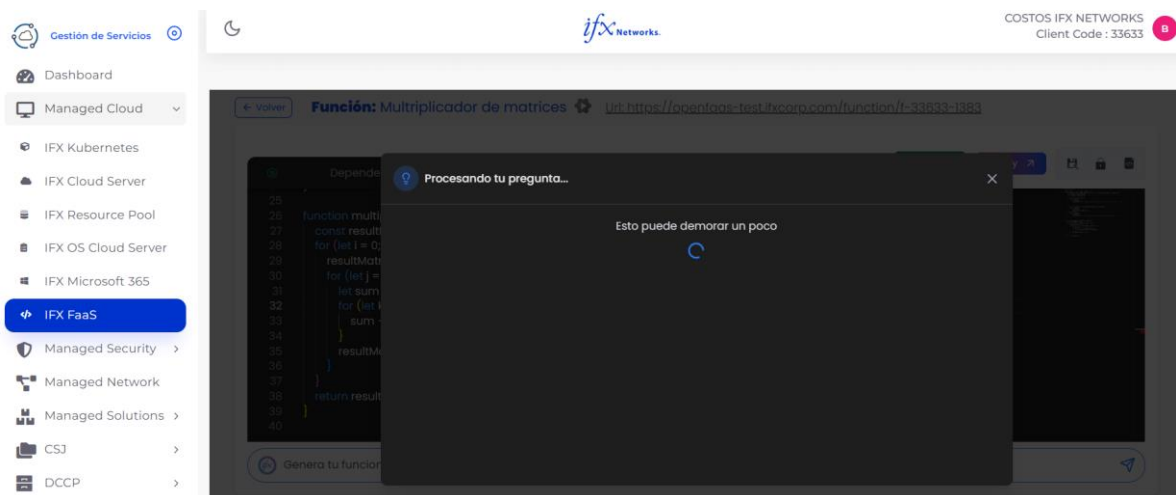


Figura 30: Respuesta a escenarios con errores - 2. Fuente: Elaboración propia.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

e. El asistente describe el error encontrado



Figura 31: Respuesta a escenarios con errores - 3. Fuente: Elaboración propia.

f. En el cuerpo el asistente sugiere la solución del código

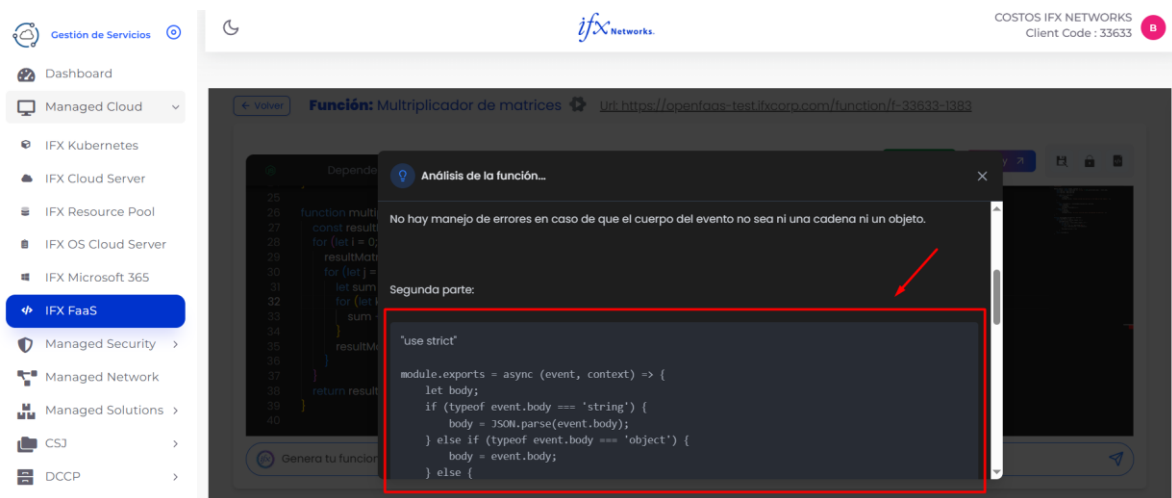


Figura 32: Respuesta a escenarios con errores - 4. Fuente: Elaboración propia.

7.3.1.4. Desplegar la función

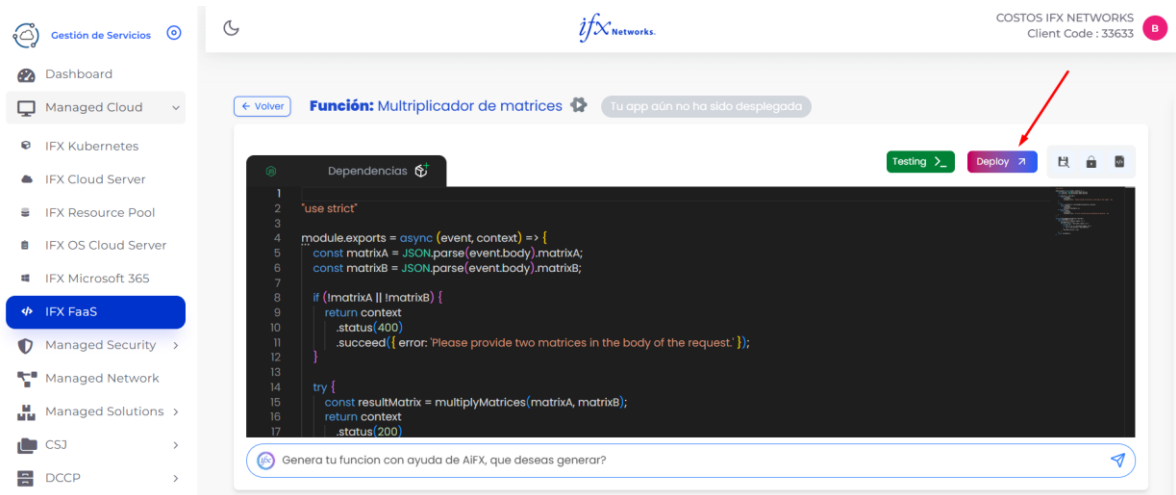


Figura 33: Despliegue de una función - 1. Fuente: Elaboración propia.

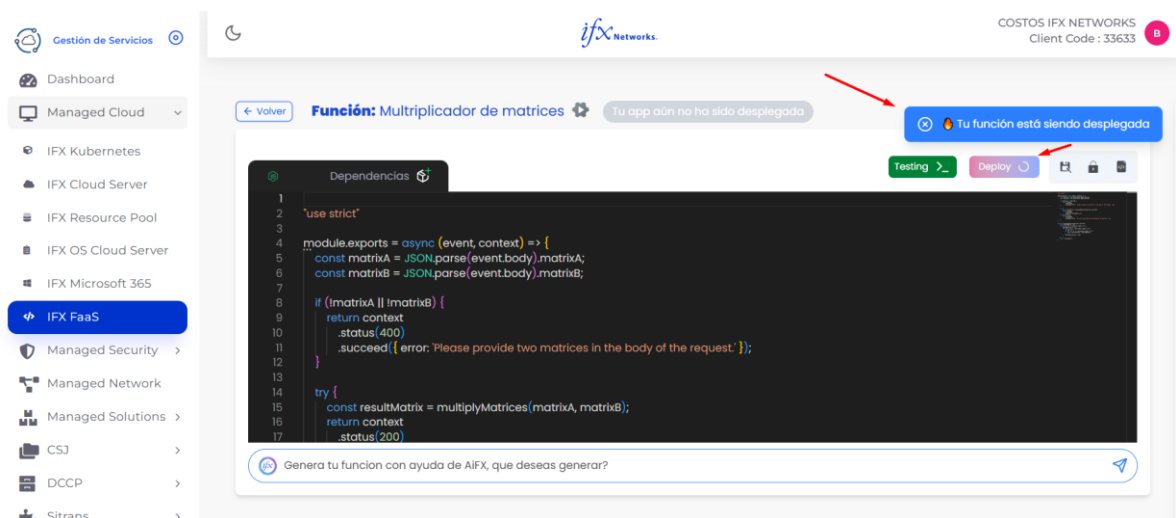


Figura 34: Despliegue de una función - 2. Fuente: Elaboración propia.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

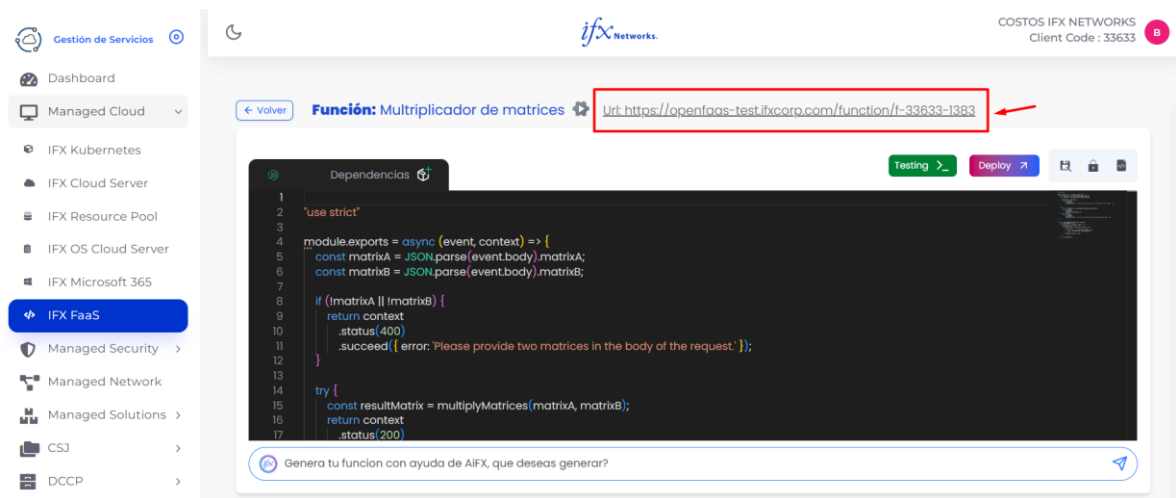


Figura 35: URL de la función desplegada. Fuente: Elaboración propia.

7.3.1.5. Validar la función desplegada

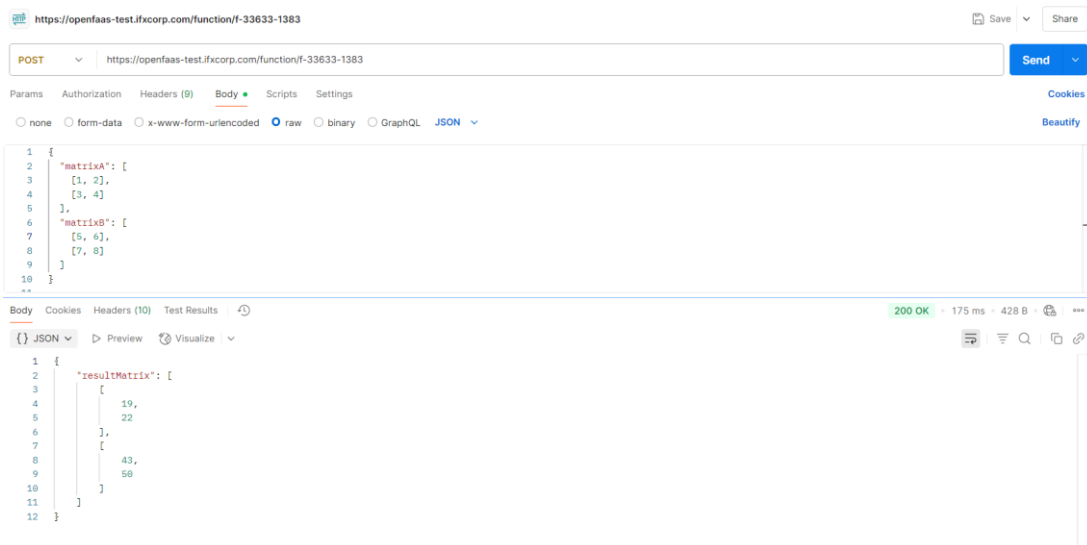


Figura 36: Prueba de consumo de la función desplegada. Fuente: Elaboración propia.

7.4. FASE DE PRUEBAS Y VALIDACIÓN DEL MODELO

7.4.1. EJECUCIÓN DE PRUEBAS

Con el objetivo de simular el tráfico y evaluar el comportamiento de rendimiento de la solución ante un escenario de alta carga, se diseñó un conjunto de pruebas en la herramienta JMeter para validar los métodos "Crear Función" y "Compilar Función".

Supuestos de las pruebas

Estas pruebas se soportan en los siguientes supuestos:

- Los equipos de la fábrica de software de IFX (DevCloud, DevNetwork, DevNetsuite, DevDigital y DevExperience) están conformados por siete desarrolladores cada uno
- Todos los desarrolladores de todos los equipos de la fábrica realizaron la creación y compilación de funciones de manera simultánea, en un periodo de tiempo inferior a un minuto.

Resultados de las pruebas

Los resultados obtenidos del ejercicio fueron los siguientes:

7.4.1.1. Información general de las pruebas de rendimiento

Se ejecutaron dos pruebas principales, con 112 peticiones en cada caso (56 por cada método). Todos los requests fueron exitosos (0% de error). Los indicadores principales se registran en la tabla 6 y en la ilustración de la figura 37 presentados a continuación.

Fecha y hora de ejecución	18/05/2025, de 7:00 pm a 7:01 pm
Cantidad total de solicitudes (divididas entre ambos endpoints)	112
Distribución:	56 peticiones para cada endpoint
Porcentaje de errores:	0% (todas las solicitudes fueron exitosas)
Indicador de estado:	100% de las transacciones fueron exitosas
Promedio global de respuesta:	1601.15 ms
Máximo observado:	5709 ms
Throughput global:	11.15 transacciones/segundo

Tabla 6. Resultados de las pruebas del modelo IFX FaaS. Fuente: Elaboración propia

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

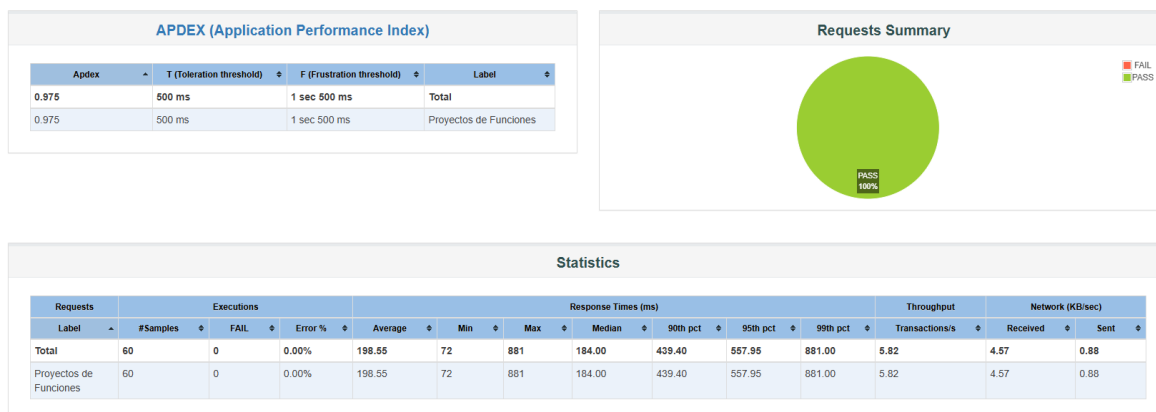


Ilustración 37. Resultados de las pruebas Modelo IFX FaaS. Fuente: Imagen tomada de los resultados Jmeter

7.4.1.2. Estadísticas por EndPoint

- Compilar Función

Tiempo de respuesta promedio:	2015.23 ms
Mediana:	1427.50 ms
Mínimo / Máximo:	370 ms / 5709 ms
Percentiles:	90th: 4374.50 ms
	95th: 4673.95 ms
	99th: 5709.00 ms
Throughput:	5.92 transacciones/segundo
Consumo de red:	1.42 KB/s (recibido), 2.58 KB/s (enviado)

Tabla 7. Estadísticas del resultado de las pruebas de Endpoints Compilar función.
Fuente: elaboración propia a partir de resultados de Jmeter.

- Crear Función

Tiempo de respuesta promedio:	1187.07 ms
Mediana:	737.00 ms
Mínimo / Máximo:	240 ms / 4074 ms
Percentiles:	90th: 3474.80 ms
	95th: 3679.60 ms
	99th: 4074.00 ms
Throughput:	5.80 transacciones/segundo
Consumo de red:	3.69 KB/s (recibido), 3.26 KB/s (enviado)

Tabla 8. Estadísticas del resultado de las pruebas de Endpoints Crear función. Fuente: elaboración propia a partir de resultados de Jmeter.

7.4.1.3. Evaluación de los Tiempos de Respuesta de los Métodos

En la figura 38 se observa el comportamiento de los tiempos de respuesta promedio para ambos métodos durante la prueba. El eje X representa la línea de tiempo comprendida entre las 19:00 y 19:01, y el eje Y indica los tiempos de respuesta en milisegundos. Se puede evidenciar que ambos métodos presentan una tendencia creciente en los tiempos de respuesta a medida que aumentan el número de peticiones, sin embargo, el método "Compile Función" alcanza los 2700 ms, el cual es mayor que el método "Crear Función" que, aunque también muestra crecimiento, es inferior, debido a que se acerca a los 2100 ms.

Este comportamiento refleja una degradación progresiva del rendimiento a medida que aumenta la carga para ambos métodos, esto se da porque el proceso realiza conexión y escritura sobre la base de datos. Sin embargo, ninguna de las dos operaciones presenta errores en su ejecución, pues todos los requests fueron exitosos (0% de error).

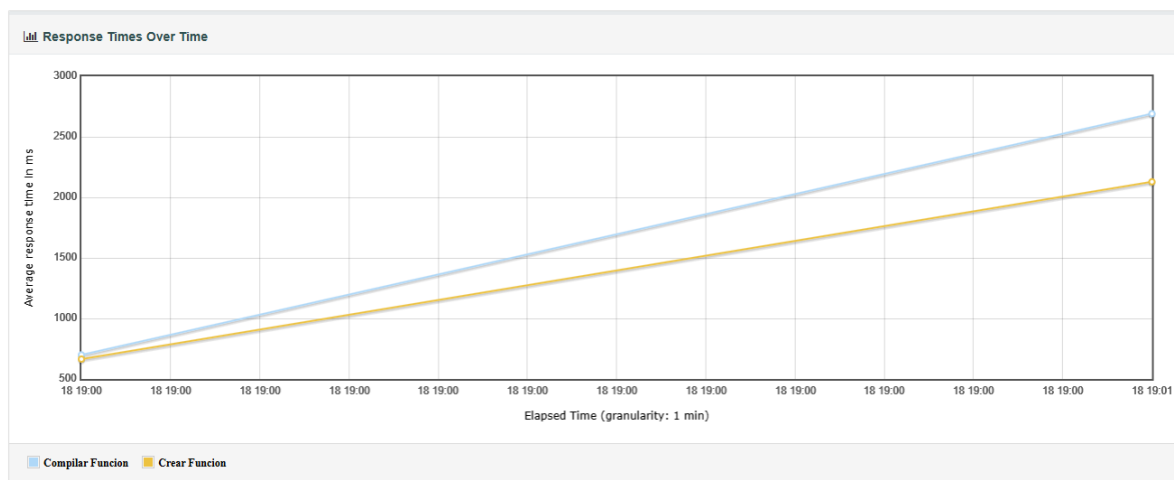


Figura 38. Tiempos de respuesta a lo largo de la prueba. Fuente: Herramienta jmeter

7.4.1.4. Hilos activos a lo largo de la prueba

En la figura 39, se observa el comportamiento a lo largo de la prueba de la evolución del número de hilos (threads) activos por equipo durante las pruebas de carga ejecutadas, en las cuales se simula la actividad concurrente de los diferentes grupos de desarrollo de la fábrica de software de IFX Networks.

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

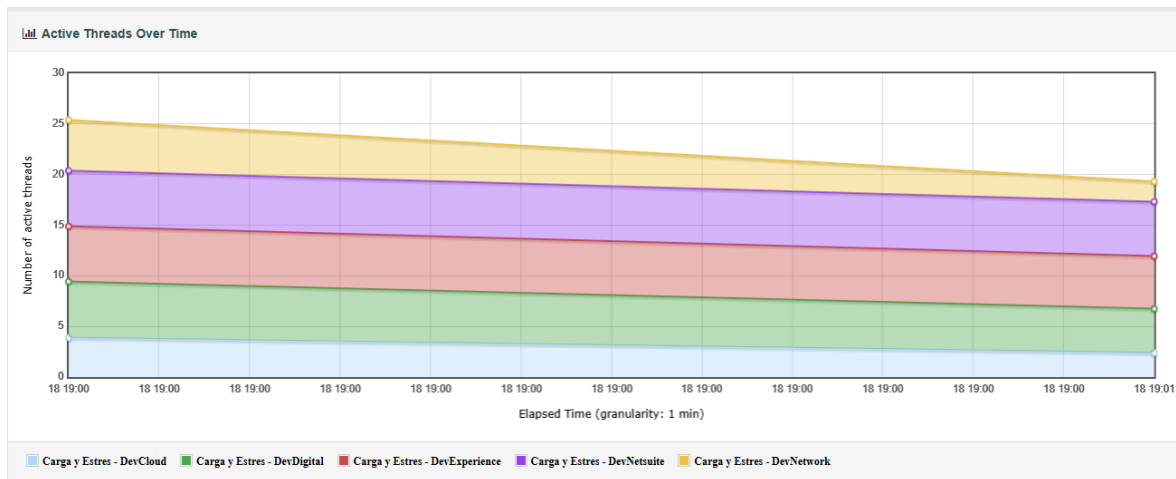


Figura 39. Hilos activos a lo largo de la prueba. Fuente: Herramienta Jmeter

El eje X representa la línea de tiempo comprendida entre las 19:00 y 19:01, y el eje Y muestra la cantidad de hilos activos simulando usuarios recurrentes, en este eje cada curva de colores representa un equipo de desarrollo de IFX, así:

- Azul claro: DevCloud
- Verde: DevDigital
- Rojo: DevExperience
- Morado: DevNetsuite
- Amarillo: DevNetwork

El comportamiento del modelo muestra que los equipos comienzan con una carga inicial alta al inicio de la prueba (19:00) y mantienen un comportamiento decreciente constante cada vez que se liberan los hilos en el proceso de ejecución, finalizando a las 19:01. El equipo DevNetwork (amarillo) tuvo el mayor número de hilos activos, iniciando con 25 y finalizando por debajo de los demás, quienes mantienen niveles proporcionales a su tamaño simulado (7 hilos cada uno), tal y como se planteó en el diseño de la prueba.

Esta distribución valida que se ejecutaron las pruebas bajo un escenario donde todos los desarrolladores de los cinco equipos realizaron tareas de manera simultáneamente, generando una carga distribuida y sostenida durante el minuto de la prueba. En la figura 39 también se puede evidenciar que la simulación de concurrencia por parte de los equipos de desarrollo es representada con una carga realista que evalúa el comportamiento de los endpoints de IFX FaaS en un escenario controlado, mostrando una reducción progresiva en los hilos activos. Esto refleja la finalización de las tareas conforme terminan sus ciclos sin interrupciones abruptas ni errores de ejecución.

4. CONCLUSIONES

A lo largo del desarrollo de este proyecto, se identificaron y evaluaron distintas alternativas para el diseño de una arquitectura que habilite el uso de servicios Serverless en la nube privada de IFX, permitiendo el diseño de un modelo que busca generar un equilibrio entre escalabilidad, desacoplamiento y facilidad de mantenimiento. La evaluación comparativa de los enfoques permitió tomar decisiones sobre la adopción de tecnologías y patrones que mejor se ajusten al contexto de la compañía.

La definición de protocolos y procesos de seguridad para las fases de autenticación y autorización es clave al momento de garantizar integridad y confidencialidad de las interacciones entre componentes distribuidos. Los aspectos de seguridad fueron integrados desde la fase de diseño, alineándose con las buenas prácticas de desarrollo y de seguridad y los marcos normativos de IFX.

El modelo conceptual de arquitectura asíncrona IFX FaaS demostró ser viable para entornos donde la carga de trabajo es dinámica y la respuesta a eventos es crítica. Su validación mediante la construcción de un prototipo funcional y pruebas de carga y estrés mediante herramientas permitió comprobar su efectividad y viabilidad de implementación sobre la infraestructura privada de IFX, o cualquier otra similar.

Se evidenció la efectividad en la integración con componentes de inteligencia artificial, mejorando la construcción de funciones y la evaluación automática sumada al ciclo de vida de las funciones, ofreciendo un valor adicional al modelo IFX FaaS.

Como proyección a futuro, se recomienda expandir el prototipo hacia un entorno de producción controlado, incluyendo herramientas de monitoreo y control de versiones de las funciones creadas. Asimismo, el modelo podría ser comercializado como un servicio cloud adicional en el portafolio de la compañía IFX para sus clientes.

5. REFERENCIAS BIBLIOGRÁFICAS

- Amazon AWS. (1 de 3 de 2025). *¿Qué es una nube privada?* Obtenido de <https://aws.amazon.com/es/what-is/private-cloud/>
- Amplifica Digital. (10 de 9 de 2024). *amplificadigital*. Obtenido de <https://amplificadigital.com.br/en/blog/serverless-the-status-of-serverless-technology-in-2024/>
- Anwar, R. W., Abdullah, T., & Pastore, F. (19 de 11 de 2021). *Firewall Best Practices for Securing Smart Healthcare Environment: A Review*. Obtenido de Applied Sciences: <https://www.mdpi.com/2076-3417/11/19/9183>
- Apache Software Foundation. (15 de 05 de 2025). *Apache OpenWhisk*. Obtenido de Apache OpenWhisk: <https://openwhisk.apache.org/>
- Billawa, P., Nicolás E., Bambhore, T., Anusha, Díaz, F., N. E., . . . Riccardo. (2022). SoK: Security of Microservice Applications: A Practitioners' Perspective on Challenges and Best Practices. *Proceedings of the 17th International Conference on Availability, Reliability and Security*, 1-10.
- Buchanan, I. (30 de 2 de 2025). *¿Qué son los contenedores como servicio?* Obtenido de <https://www.atlassian.com/es/microservices/cloud-computing/containers-as-a-service>
- Cabrera, R. (1 de 8 de 2023). *Importancia de arquitecturas asíncronas en los sistemas distribuidos*. Obtenido de <https://aws.amazon.com/es/blogs/aws-spanish/importancia-de-arquitecturas-asincronas-en-los-sistemas-distribuidos/>
- Castro, P., Ishakian, V., Muthusamy, V., & Slominski, A. (December de 2019). *The Rise of Serverless Computing*. Obtenido de Communications of the ACM: <https://cacm.acm.org/magazines/2019/12/241054-the-rise-of-serverless-computing/fulltext>
- Ćatović, A., Buzadžija, N., & Lemeš, S. (29 de 04 de 2022). *Microservice development using RabbitMQ message broker*. Obtenido de Science, Engineering and Technology: <http://www.setjournal.com/>

- Centria. (2025). *www.centria.es*. Obtenido de https://www.centria.es/wp-content/uploads/2023/10/traditional_ins.png
- Cloud, G. (2025). *https://cloud.google.com/functions/docs*. Obtenido de <https://cloud.google.com/functions/docs>
- Danave, S. (12 de 9 de 2024). *Top 5 cloud computing trends of 2024*. Obtenido de <https://www.cncf.io/blog/2024/05/03/top-5-cloud-computing-trends-of-2024/>
- ELMASRI, R., & NAVATHE, S. (2007). *Fundamentos de Sistemas de Base de Datos*. PEARSON EDUCACIÓN S.A.
- Fixed Buffer. (24 de 11 de 2020). Obtenido de <https://www.fixedbuffer.com/como-crear-imagenes-docker-multiarquitectura/>
- Fn Project. (15 de 04 de 2025). *Fn Project*. Obtenido de Fn Project: <https://fnproject.io/>
- Functions, M. A. (2025). *https://learn.microsoft.com/en-us/azure/azure-functions/functions/functions-best-practices*. Obtenido de <https://learn.microsoft.com/en-us/azure/azure-functions/functions/functions-best-practices>
- IFX Networks. (2024). *Acerca de nosotros: IFX Networks*. Obtenido de <https://www.ifxnetworks.com/nosotros/>
- IFX Networks. (2024). *Estándar codificación segura en IFX*. Bogotá.
- IFX Networks. (2024). *Política de control de cambios de desarrollo*. Bogotá.
- IFX Networks. (2024). *Política desarrollo seguro DevCenter*. Bogotá.
- IFX Networks. (2025). *Acerca de nosotros: IFX Networks*. Obtenido de <https://ifxnetworks.com/soluciones>
- Jung Kang, M., & Jihyoun, P. (2013). Analysis of the partnership network in the clean development mechanism.
- Kidd, C., & Wickramasinghe, S. (6 de 8 de 2021). *Serverless vs Function-as-a-Service (FaaS): What's The Difference?* Obtenido de bmc: <https://www.bmc.com/blogs/serverless-faas/>

Knative. (23 de 05 de 2025). *Knative Overview*. Obtenido de Knative: <https://knative.dev/docs/concepts/>

Kubernetes. (17 de 7 de 2022). Obtenido de ¿Qué es Kubernetes?: <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/>

Kubernetes. (17 de 7 de 2022). Obtenido de ¿Por qué usar contenedores?: <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/#por-qu%C3%A9-usar-contenedores>

Kurapati, L. (2024). Micro Frontend Architecture: Benefits, Challenges, and Best Practices. *International Journal for Multidisciplinary Research*, 1-4.

Lambda, A. (2025). <https://aws.amazon.com/lambda/faqs/>. Obtenido de <https://aws.amazon.com/lambda/faqs/>

Maquera Quispe, H., Ticse Capcha, R., Gómez Morales, P., & Quintana, C. (2019). MODELADO BPMN (BUSINESS PROCESS MANAGEMENT NOTATION) PARA LA GESTIÓN DE PROCESOS. <https://doi.org/10.33326/26176033.2014.18.460>.

Microsoft Learn. (30 de 2 de 2025). *Asynchronous Request-Reply pattern*. Obtenido de <https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply>

Nagaraj, C., & Sriramulu, R. (2019). ON-PREMISES INFRASTRUCTURE FOR SERVERLESS COMPUTING. *Dell Technologies Proven Professional Knowledge Sharing*, 8.

okta. (28 de 8 de 2024). *What Is BaaS (Backend as a Service)? Definition and Usage*. Obtenido de <https://www.okta.com/identity-101/baas-backend-as-a-service/>

OpenFaaS Ltd. (1 de 05 de 2025). *OPENFAAS*. Obtenido de OPENFAAS: <https://www.openfaas.com/>

RedHat. (24 de 2 de 2025). Obtenido de <https://www.redhat.com/es/topics/virtualization/what-is-a-hypervisor>

RedHat. (22 de 2 de 2025). *RedHat*. Obtenido de <https://www.redhat.com/es/topics/virtualization/what-is-virtualization>

- Rodríguez Domínguez, J. (2020). *"Serverless Computing", Funciones como Servicio (FaaS) para el soporte de cargas computacionales en la nube*. Obtenido de <https://m.riunet.upv.es/bitstream/handle/10251/159165/Rodriguez%20-%20Serverless%20Computing%2C%20Funciones%20como%20Servicio%20%28FaaS%29%20para%20el%20soporte%20de%20cargas%20comp....pdf>
- Rumbaugh, J., Jacobson, I., & Booch, G. (2000). *El lenguaje UNificado de Modelado Manual de referencia*. Madrid: Pearson Education S.A.
- Sanfrutos, E. (Junio de 2020). *Aplicación de tecnologías y arquitecturas serverless para el desarrollo de soluciones IoT*. España.
- Susnjara, S., & Smalley, I. (26 de 2 de 2025). *¿Qué es Docker?* Obtenido de <https://www.ibm.com/es-es/think/topics/docker>
- TI, F. (2018). *Ingeniería de requisitos IREB*. Bogota: Intelligent Training.
- Weins, K. (2020, Mayo 21). *Cloud Computing Trends: 2020 State of the Cloud Report*. Retrieved from <https://www.flexera.com/blog/industry-trends/trend-of-cloud-computing-2020/>
- Weins, K. (2023, Septiembre 9). *Cloud computing trends and statistics: Flexera 2023 State of the Cloud Report*. Retrieved from <https://www.flexera.com/blog/finops/cloud-computing-trends-flexera-2023-state-of-the-cloud-report/>
- Winzinger, S., & Wirtz, G. (2019). *Model-Based Analysis of Serverless Applications*. IEEE.
- Wmware. (24 de 2 de 2025). *Virtualización*. Obtenido de <https://www.vmware.com/solutions/cloud-infrastructure/virtualization>
- WoSC6. (2020). *Sixth International Workshop on Serverless Computing (WoSC6) 2020*. Obtenido de <https://www.serverlesscomputing.org/wosc6/>
- Zettler, K. (30 de 2 de 2025). Obtenido de <https://www.atlassian.com/es/microservices/microservices-architecture/distributed-architecture>

Diseño de modelo arquitectónico asíncrono para la implementación de servicios de computación sin servidor en la infraestructura de nube privada de IFX Networks

Zhao, H., Benomar, Z., Pfandzelter, T., & Georgantas, N. (2022). Supporting Multi-Cloud in Serverless Computing. *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*, 285-290.